

# PES UNIVERSITY, Bangalore

Established under Karnataka Act No. 16 of 2013)

## Department of Computer Science & Engineering

### UE24CS251A: DIGITAL DESIGN AND COMPUTER ORGANIZATION

#### UNIT 1:

#### BASIC THEOREMS AND PROPERTIES OF BOOLEAN ALGEBRA:

Binary logic is used in all of today's digital computers and devices, the cost of the circuits that implement it is an important factor addressed by designers—be they computer engineers, electrical engineers, or computer scientists. Finding simpler and cheaper, but equivalent, realizations of a circuit can reap huge payoffs in reducing the overall cost of the design. Mathematical methods that simplify circuits rely primarily on Boolean algebra. Therefore, this chapter provides a basic vocabulary and a brief foundation in Boolean algebra that will enable you to optimize simple circuits and to understand the purpose of algorithms used by software tools to optimize complex circuits involving millions of logic gates.

The Huntington postulates were listed in pairs and designated by part (a) and part (b). One part may be obtained from the other if the binary operators and the identity elements are interchanged. This important property of Boolean algebra is called the duality principle and states that every algebraic expression deducible from the postulates of Boolean algebra remains valid if the operators and identity elements are interchanged. In a two-valued Boolean algebra, the identity elements and the elements of the set B are the same: 1 and 0. The duality principle has many applications. If the dual of an algebraic expression is desired, we simply interchange OR and AND operators and replace 1's by 0's and 0's by 1's.

#### *Postulates and Theorems of Boolean Algebra*

|                           |     |                             |     |                           |
|---------------------------|-----|-----------------------------|-----|---------------------------|
| Postulate 2               | (a) | $x + 0 = x$                 | (b) | $x \cdot 1 = x$           |
| Postulate 5               | (a) | $x + x' = 1$                | (b) | $x \cdot x' = 0$          |
| Theorem 1                 | (a) | $x + x = x$                 | (b) | $x \cdot x = x$           |
| Theorem 2                 | (a) | $x + 1 = 1$                 | (b) | $x \cdot 0 = 0$           |
| Theorem 3, involution     |     | $(x')' = x$                 |     |                           |
| Postulate 3, commutative  | (a) | $x + y = y + x$             | (b) | $xy = yx$                 |
| Theorem 4, associative    | (a) | $x + (y + z) = (x + y) + z$ | (b) | $x(yz) = (xy)z$           |
| Postulate 4, distributive | (a) | $x(y + z) = xy + xz$        | (b) | $x + yz = (x + y)(x + z)$ |
| Theorem 5, DeMorgan       | (a) | $(x + y)' = x'y'$           | (b) | $(xy)' = x' + y'$         |
| Theorem 6, absorption     | (a) | $x + xy = x$                | (b) | $x(x + y) = x$            |

#### Operator Precedence:

The operator precedence for evaluating Boolean expressions is (1) parentheses, (2) NOT, (3) AND, and (4) OR. In other words, expressions inside parentheses must be evaluated before all other operations. The next operation that holds precedence is the complement, and then follows the AND and, finally, the OR. As an example, consider the truth table for one of DeMorgan's theorems. The left side of the expression is  $(x + y)'$ . Therefore, the expression inside the parentheses is evaluated first and the result then complemented. The right side of the expression

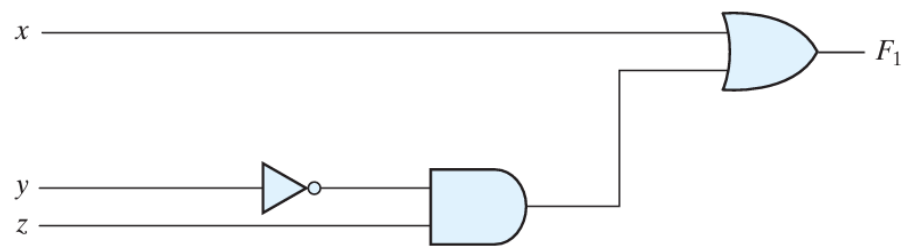
is  $\overline{xy}$ , so the complement of  $x$  and the complement of  $y$  are both evaluated first and the result is then ANDed. Note that in ordinary arithmetic, the same precedence holds (except for the complement) when multiplication and addition are replaced by AND and OR, respectively.

### Boolean Functions:

Boolean algebra is an algebra that deals with binary variables and logic operations. A Boolean function described by an algebraic expression consists of binary variables, the constants 0 and 1, and the logic operation symbols. For a given value of the binary variables, the function can be equal to either 1 or 0. As an example, consider the Boolean function  $F_1 = x + yz$ . The function  $F_1$  is equal to 1 if  $x$  is equal to 1 or if both  $y$  and  $z$  are equal to 1.  $F_1$  is equal to 0 otherwise. The complement operation dictates that when  $y = 1$ ,  $y = 0$ . Therefore,  $F_1 = 1$  if  $x = 1$  or if  $y = 0$  and  $z = 1$ . A Boolean function expresses the logical relationship between binary variables and is evaluated by determining the binary value of the expression for all possible values of the variables. A Boolean function can be represented in a truth table. The number of rows in the truth table is  $2^n$ , where  $n$  is the number of variables in the function. The binary combinations for the truth table are obtained from the binary numbers by counting from 0 through  $2^n - 1$ . Table 2.2 shows the truth table for the function  $F_1$ . There are eight possible binary combinations for assigning bits to the three variables  $x$ ,  $y$ , and  $z$ . The column labeled  $F_1$  contains either 0 or 1 for each of these combinations. The table shows that the function is equal to 1 when  $x = 1$  or when  $yz = 01$  and is equal to 0 otherwise. A Boolean function can be transformed from an algebraic expression into a circuit diagram composed of logic gates connected in a particular structure. The logic-circuit diagram (also called a schematic) for  $F_1$  is shown in Fig 2.1. There is an inverter for input  $y$  to generate its complement. There is an AND gate for the term  $yz$  and an OR gate

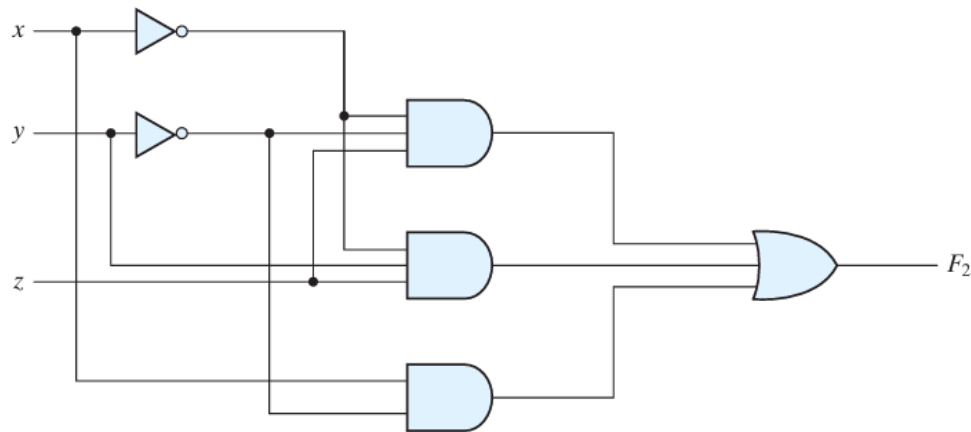
**Table 2.2**  
*Truth Tables for  $F_1$  and  $F_2$*

| $x$ | $y$ | $z$ | $F_1$ | $F_2$ |
|-----|-----|-----|-------|-------|
| 0   | 0   | 0   | 0     | 0     |
| 0   | 0   | 1   | 1     | 1     |
| 0   | 1   | 0   | 0     | 0     |
| 0   | 1   | 1   | 0     | 1     |
| 1   | 0   | 0   | 1     | 1     |
| 1   | 0   | 1   | 1     | 1     |
| 1   | 1   | 0   | 1     | 0     |
| 1   | 1   | 1   | 1     | 0     |

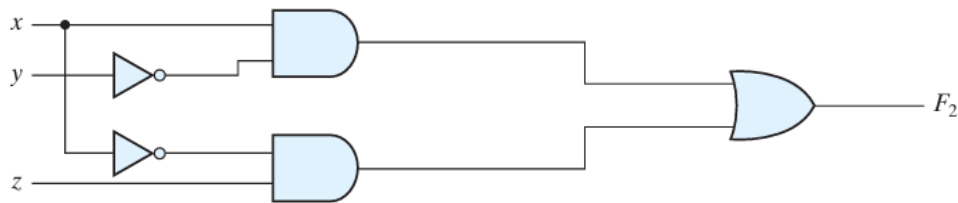


**FIGURE 2.1**  
Gate implementation of  $F_1 = x + y'z$

that combines  $x$  with  $yz$ . In logic-circuit diagrams, the variables of the function are taken as the inputs of the circuit and the binary variable  $F_1$  is taken as the output of the circuit. The schematic expresses the relationship between the output of the circuit and its inputs. Rather than listing each combination of inputs and outputs, it indicates how to compute the logic value of each output from the logic values of the inputs. There is only one way that a Boolean function can be represented in a truth table. However, when the function is in algebraic form, it can be expressed in a variety of ways, all of which have equivalent logic. The particular expression used to represent the function will dictate the interconnection of gates in the logic-circuit diagram. Conversely, the interconnection of gates will dictate the logic expression. Here is a key fact that motivates our use of Boolean algebra: By manipulating a Boolean expression according to the rules of Boolean algebra, it is sometimes possible to obtain a simpler expression for the same function and thus reduce the number of gates in the circuit and the number of inputs to the gate. Designers are motivated to reduce the complexity and number of gates because their effort can significantly reduce the cost of a circuit. Consider, for example, the following Boolean function:  $F_2 = xyz + xyz + xy$ . A schematic of an implementation of this function with logic gates is shown in Fig. 2.2 (a). Input variables  $x$  and  $y$  are complemented with inverters to obtain  $x'$  and  $y'$ . The three terms in the expression are implemented with three AND gates. The OR gate forms the logical OR of the three terms. The truth table for  $F_2$  is listed in Table 2.2. The function is equal to 1 when  $xyz = 001$  or  $011$  or when  $xy = 10$  (irrespective of the value of  $z$ ) and is equal to 0 otherwise. This set of conditions produces four 1's and four 0's for  $F_2$ . Now consider the possible simplification of the function by applying some of the identities of Boolean algebra:  $F_2 = xyz + xyz + xy = xz(y + y') + xy = xz + xy$ . The function is reduced to only two terms and can be implemented with gates as shown in Fig. 2.2 (b). It is obvious that the circuit in (b) is simpler than the one in (a), yet both implement the same function. By means of a truth table, it is possible to verify that the two expressions are equivalent. The simplified expression is equal to 1 when  $xz = 01$  or when  $xy = 10$ . This produces the same four 1's in the truth table. Since both expressions produce the same truth table, they are equivalent. Therefore, the two circuits have the same outputs for all possible binary combinations of inputs of the three variables. Each circuit implements the same identical function, but the one with fewer gates and fewer inputs to gates is preferable because it requires fewer wires and components. In general, there are many equivalent representations of a logic function. Finding the most economic representation of the logic is an important design task.



(a)  $F_2 = x'y'z + x'yz + xy'$



(b)  $F_2 = xy' + x'z$

**FIGURE 2.2**

Implementation of Boolean function  $F_2$  with gates

**Complement of a Function** The complement of a function  $F$  is  $\bar{F}$  and is obtained from an interchange of 0's for 1's and 1's for 0's in the value of  $F$ . The complement of a function may be derived algebraically through DeMorgan's theorems, listed in Table 2.1 for two variables. DeMorgan's theorems can be extended to three or more variables. The three-variable form of the first DeMorgan's theorem is derived as follows, from postulates and theorems listed in Table 2.1 :  $(A + B + C)' = (A + x)'$  let  $B + C = x = Ax$  by theorem 5(a) (DeMorgan)  $= A(B + C)'$  substitute  $B + C = x = A(BC)'$  by theorem 5(a) (DeMorgan)  $= ABC'$  by theorem 4(b) (associative) DeMorgan's theorems for any number of variables resemble the two-variable case in form and can be derived by successive substitutions similar to the method used in the preceding derivation. These theorems can be generalized as follows:  $(A + B + C + D + \dots + F)' = ABCD\bar{C}\bar{F}$   $(ABCD\bar{C}\bar{F})' = A + B + C + D + \dots + F$  The generalized form of DeMorgan's theorems states that the complement of a function is obtained by interchanging AND and OR operators and complementing each literal.

## Canonical and Standard forms

Min terms and Max terms:

A binary variable may appear either in its normal form ( $x$ ) or in its complement form ( $\bar{x}$ ). Now consider two binary variables  $x$  and  $y$  combined with an AND operation. Since each variable may appear in either form, there are four possible combinations:  $xy$ ,  $x\bar{y}$ ,  $\bar{x}y$ , and  $\bar{x}\bar{y}$ . Each of these four AND terms is called a minterm, or a standard product. In a similar manner,  $n$

variables can be combined to form  $2^n$  minterms. The  $2^n$  different min terms may be determined by a method similar to the one shown in Table 2.3 for three variables. The binary numbers from 0 to  $2^n-1$  are listed under the  $n$  variables. In a similar fashion,  $n$  variables forming an OR term, with each variable being primed or unprimed, provide  $2^n$  possible combinations, called maxterms, or standard sums. The eight maxterms for three variables, together with their symbolic designations.

A Boolean function can be expressed algebraically from a given truth table by forming a minterm for each combination of the variables that produces a 1 in the function and then taking the OR of all those terms.

**Table 2.3**  
*Minterms and Maxterms for Three Binary Variables*

| <b>x</b> | <b>y</b> | <b>z</b> | <b>Minterms</b> |                    | <b>Maxterms</b> |                    |
|----------|----------|----------|-----------------|--------------------|-----------------|--------------------|
|          |          |          | <b>Term</b>     | <b>Designation</b> | <b>Term</b>     | <b>Designation</b> |
| 0        | 0        | 0        | $x'y'z'$        | $m_0$              | $x + y + z$     | $M_0$              |
| 0        | 0        | 1        | $x'y'z$         | $m_1$              | $x + y + z'$    | $M_1$              |
| 0        | 1        | 0        | $x'yz'$         | $m_2$              | $x + y' + z$    | $M_2$              |
| 0        | 1        | 1        | $x'yz$          | $m_3$              | $x + y' + z'$   | $M_3$              |
| 1        | 0        | 0        | $xy'z'$         | $m_4$              | $x' + y + z$    | $M_4$              |
| 1        | 0        | 1        | $xy'z$          | $m_5$              | $x' + y + z'$   | $M_5$              |
| 1        | 1        | 0        | $xyz'$          | $m_6$              | $x' + y' + z$   | $M_6$              |
| 1        | 1        | 1        | $xyz$           | $m_7$              | $x' + y' + z'$  | $M_7$              |

**Table 2.4**  
*Functions of Three Variables*

| <b>x</b> | <b>y</b> | <b>z</b> | <b>Function <math>f_1</math></b> | <b>Function <math>f_2</math></b> |
|----------|----------|----------|----------------------------------|----------------------------------|
| 0        | 0        | 0        | 0                                | 0                                |
| 0        | 0        | 1        | 1                                | 0                                |
| 0        | 1        | 0        | 0                                | 0                                |
| 0        | 1        | 1        | 0                                | 1                                |
| 1        | 0        | 0        | 1                                | 0                                |
| 1        | 0        | 1        | 0                                | 1                                |
| 1        | 1        | 0        | 0                                | 1                                |
| 1        | 1        | 1        | 1                                | 1                                |

The procedure for obtaining the product of maxterms directly from the truth table is as follows: Form a maxterm for each combination of the variables that produces a 0 in the function, and

then form the AND of all those maxterms. Boolean functions expressed as a sum of minterms or product of maxterms are said to be in canonical form

Sum of Minterms:

The minterms whose sum defines the Boolean function are those which give the 1's of the function in a truth table. Since the function can be either 1 or 0 for each minterm, and since there are  $2^n$  minterms, one can calculate all the functions that can be formed with  $n$  variables to be  $2^{2^n}$ . It is sometimes convenient to express a Boolean function in its sum-of-minterms form. If the function is not in this form, it can be made so by first expanding the expression into a sum of AND terms. Each term is then inspected to see if it contains all the variables. If it misses one or more variables, it is ANDed with an expression such as  $x + x'$ , where  $x$  is one of the missing variables.

An alternative procedure for deriving the minterms of a Boolean function is to obtain the truth table of the function directly from the algebraic expression and then read the minterms from the truth table.

**Table 2.5**  
*Truth Table for  $F = A + B'C$*

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

### Product of Maxterms

Each of the  $2^{2^n}$  functions of  $n$  binary variables can be also expressed as a product of maxterms. To express a Boolean function as a product of maxterms, it must first be brought into a form of OR terms. This may be done by using the distributive law,  $x+yz=(x+y)(x+z)$ . Then any missing variable  $x$  in each OR term is ORed with  $xx'$

A convenient way to express this function is as follows:  $F(x, y, z) = A = \pi(0, 2, 4, 5)$

### Conversion between Canonical Forms

The complement of a function expressed as the sum of minterms equals the sum of min terms missing from the original function. This is because the original function is expressed by those minterms which make the function equal to 1, whereas its complement is a 1 for those minterms for which the function is a 0. As an example, consider the function

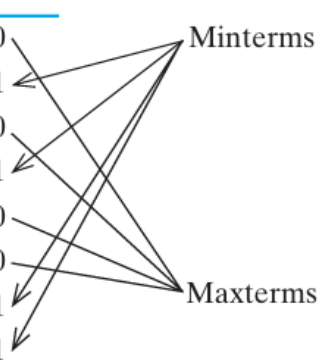
$F(A, B, C) = \sum (1, 4, 5, 6, 7)$  This function has a complement that can be expressed as

$F(A, B, C) = \sum(0, 2, 3) = m_0 + m_2 + m_3$  Now, if we take the complement of  $F$  by DeMorgan's theorem, we obtain  $F$  in a different form:  $F = (m_0 + m_2 + m_3)$  i.e.  $M_0M_2M_3 = \pi(0, 2, 3)$

That is, the maxterm with subscript  $j$  is a complement of the minterm with the same subscript  $j$  and vice versa.

**Table 2.6**  
*Truth Table for  $F = xy + x'z$*

| $x$ | $y$ | $z$ | $F$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 1   |
| 0   | 1   | 0   | 0   |
| 0   | 1   | 1   | 1   |
| 1   | 0   | 0   | 0   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 1   |
| 1   | 1   | 1   | 1   |



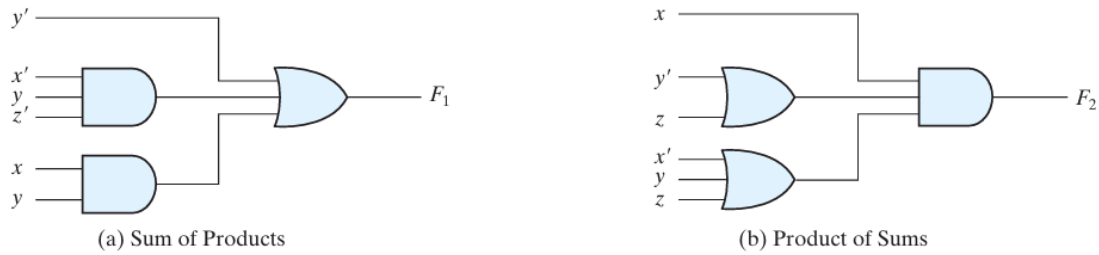
### Standard Forms:

The two canonical forms of Boolean algebra are basic forms that one obtains from reading a given function from the truth table. These forms are very seldom the ones with the least number of literals, because each minterm or maxterm must contain, by definition, all the variables, either complemented or uncomplemented. Another way to express Boolean functions is in standard form. In this configuration, the terms that form the function may contain one, two, or any number of literals. There are two types of standard forms: the sum of products and products of sums. The sum of products is a Boolean expression containing AND terms, called product terms, with one or more literals each. The sum denotes the ORing of these terms. An example of a function expressed as a sum of products is

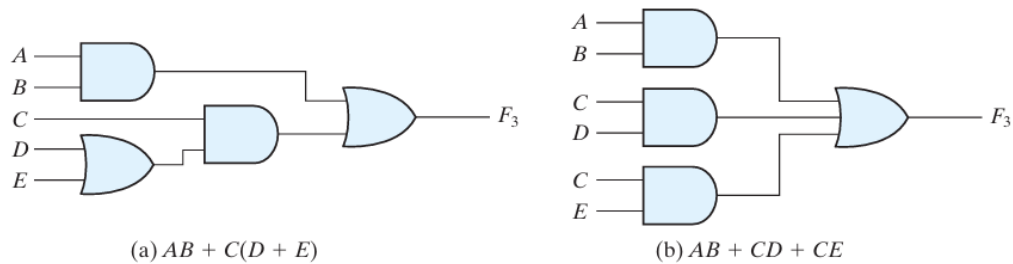
$$F1 = y' + xy + x'yz'$$

The expression has three product terms, with one, two, and three literals. Their sum is, in effect, an OR operation.

The logic diagram of a sum-of-products expression consists of a group of AND gates followed by a single OR gate. This configuration pattern is shown in Fig. 2.3 (a). Each product term requires an AND gate, except for a term with a single literal. The logic sum is formed with an OR gate whose inputs are the outputs of the AND gates and the single literal. It is assumed that the input variables are directly available in their complements, so inverters are not included in the diagram. This circuit configuration is referred to as a two-level implementation.



**FIGURE 2.3**  
Two-level implementation



**FIGURE 2.4**  
Three- and two-level implementation

A Boolean function may be expressed in a nonstandard form. For example, the function  $F_3 = AB + C(D + E)$  is neither in sum-of-products nor in product-of-sums form. The implementation of this expression is shown in Fig. 2.4 (a) and requires two AND gates and two OR gates. There are three levels of gating in this circuit. It can be changed to a standard form by using the distributive law to remove the parentheses:  $F_3 = AB + C(D + E) = AB + CD + CE$

In general, a two-level implementation is preferred because it produces the least amount of delay through the gates when the signal propagates from the inputs to the output. However, the number of inputs to a given gate might not be practical.

### Gate-level minimization:

#### K-Maps

Gate-level minimization is the design task of finding an optimal gate-level implementation of the Boolean functions describing a digital circuit. This task is well understood, but is difficult to execute by manual methods when the logic has more than a few inputs. Fortunately, computer-based logic synthesis tools can minimize a large set of Boolean equations efficiently and quickly.

The complexity of the digital logic gates that implement a Boolean function is directly related to the complexity of the algebraic expression from which the function is implemented. Although the truth table representation of a function is unique, when it is expressed algebraically it can appear in many different, but equivalent, forms. The map method presented here provides a simple, straightforward procedure for minimizing Boolean functions. This method may be regarded as a pictorial form of a truth table. The map method is also known as the Karnaugh map or K-map. A K-map is a diagram made up of squares, with each square

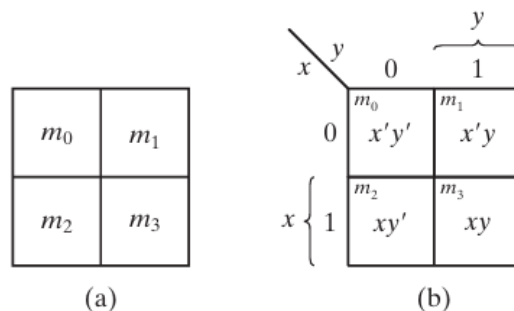


representing one minterm of the function that is to be minimized. Since any Boolean function can be expressed as a sum of minterms, it follows that a Boolean function is recognized graphically in the map from the area enclosed by those squares whose minterms are included in the function. In fact, the map presents a visual diagram of all possible ways a function may be expressed in standard form. By recognizing various patterns, the user can derive alternative algebraic expressions for the same function, from which the simplest can be selected.

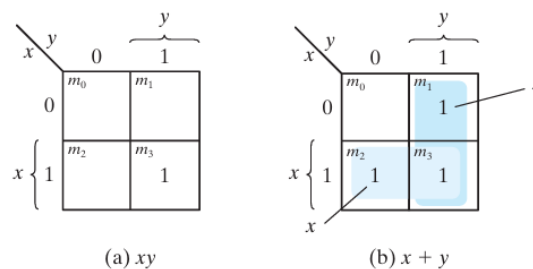
### 2 variable K Map:

The two-variable map is shown in Fig. 3.1 (a). There are four minterms for two variables; hence, the map consists of four squares, one for each minterm. The map is redrawn in (b) to show the relationship between the squares and the two variables  $x$  and  $y$ . The 0 and 1 marked in each row and column designate the values of variables. Variable  $x$  appears primed in row 0 and unprimed in row 1. Similarly,  $y$  appears primed in column 0 and unprimed in column 1. If we mark the squares whose minterms belong to a given function, the two-variable map becomes another useful way to represent any one of the 16 Boolean functions of two variables. As an example, the function  $xy$  is shown in Fig. 3.2 (a). Since  $xy$  is equal to  $m_3$ , a 1 is placed inside the square that belongs to  $m_3$ . Similarly, the function  $x + y$  is represented in the map of Fig. 3.2 (b) by three squares marked with 1's. These squares are found from the minterms of the function:

$$m_1 + m_2 + m_3 = x'y + xy' + xy = x + y$$



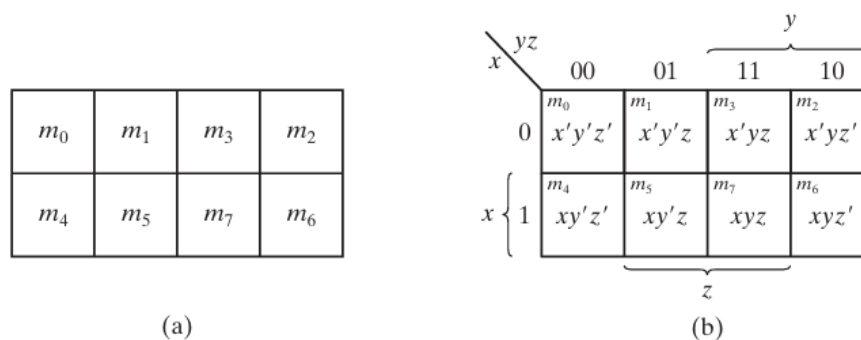
**FIGURE 3.1**  
Two-variable K-map



**FIGURE 3.2**  
Representation of functions in the map

### Three-Variable K-Map:

A three-variable K-map is shown in Fig. 3.3 . There are eight minterms for three binary variables; therefore, the map consists of eight squares. Note that the minterms are arranged, not in a binary sequence, but in a sequence similar to the Gray code ( Table 1.6 ). The characteristic of this sequence is that only one bit changes in value from one adjacent column to the next. The map drawn in part (b) is marked with numbers in each row and each column to show the relationship between the squares and the three variables. For example, the square assigned to  $m_5$  corresponds to row 1 and column 01. When these two numbers are concatenated, they give the binary number 101, whose decimal equivalent is 5. Each cell of the map corresponds to a unique minterm, so another way of looking at square  $m_5=xyz$  is to consider it to be in the row marked  $x$  and the column belonging to  $yz$  (column 01). Note that there are four squares in which each variable is equal to 1 and four in which each is equal to 0.



**FIGURE 3.3**  
Three-variable K-map

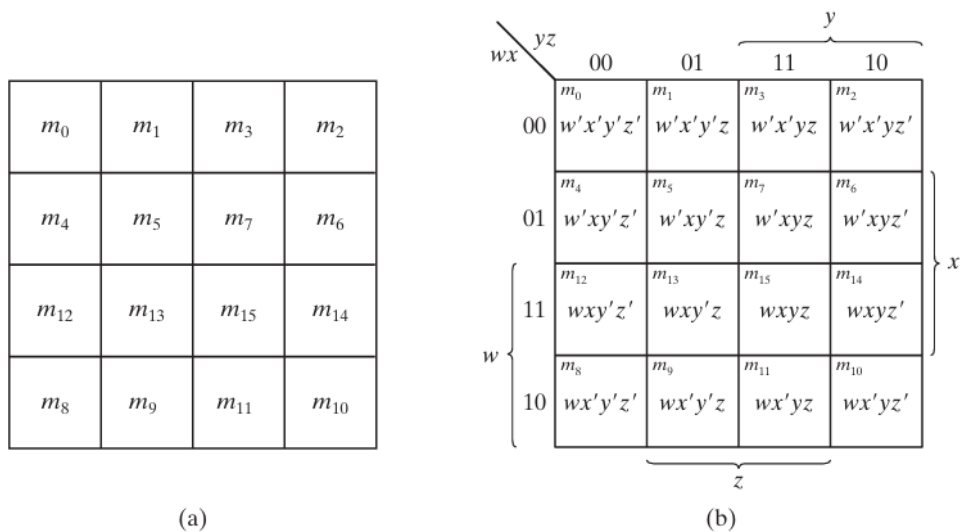
To understand the usefulness of the map in simplifying Boolean functions, we must recognize the basic property possessed by adjacent squares: Any two adjacent squares in the map differ by only one variable, which is primed in one square and unprimed in the other. The number of adjacent squares that may be combined must always represent a number that is a power of two, such as 1, 2, 4, and 8. As more adjacent squares are combined, we obtain a product term with fewer literals.

- One square represents one minterm, giving a term with three literals.
- Two adjacent squares represent a term with two literals.
- Four adjacent squares represent a term with one literal.
- Eight adjacent squares encompass the entire map and produce a function that is always equal to 1.

4 variable Kmap:

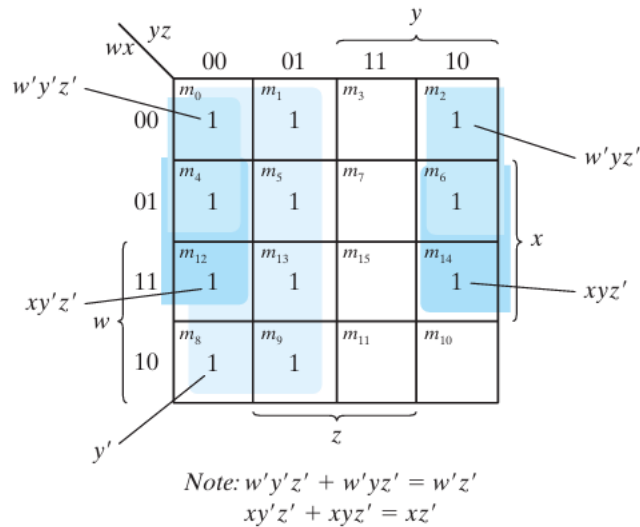
The map for Boolean functions of four binary variables ( $w, x, y, z$ ) is shown in Fig. 3.8 . In Fig. 3.8(a) are listed the 16 minterms and the squares assigned to each. In Fig. 3.8(b), the map is redrawn to show the relationship between the squares and the four variables. The rows and columns are numbered in a Gray code sequence, with only one digit changing value between two adjacent rows or columns. The minterm corresponding to each square can be obtained from the concatenation of the row number with the column number. For example, the numbers of the third row (11) and the second column (01), when concatenated, give the binary number 1101, the binary equivalent of decimal 13. Thus, the square in the third row and second column

represents minterm  $m_{13}$ . The map minimization of four-variable Boolean functions is similar to the method used to minimize three-variable functions. Adjacent squares are defined to be squares next to each other. In addition, the map is considered to lie on a surface with the top and bottom edges, as well as the right and left edges, touching each other to form adjacent squares. For example,  $m_0$  and  $m_2$  form adjacent squares, as do  $m_3$  and  $m_{11}$ . The combination of adjacent squares that is useful during the simplification process is easily determined from inspection of the four-variable map: One square represents one minterm, giving a term with four literals. Two adjacent squares represent a term with three literals. Four adjacent squares represent a term with two literals. Eight adjacent squares represent a term with one literal. Sixteen adjacent squares produce a function that is always equal to 1. No other combination of squares can simplify the function. The next two examples show the procedure used to simplify four-variable Boolean functions.



**FIGURE 3.8**  
Four-variable map

$$F = y' + w'z' + xz'$$

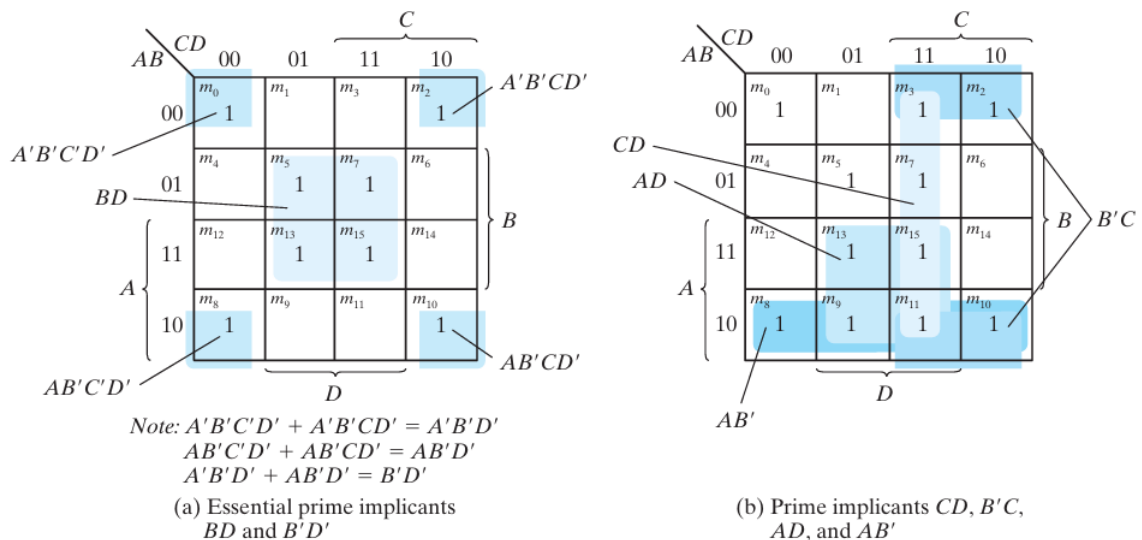


**FIGURE 3.9**

Map for Example 3.5,  $F(w, x, y, z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14) = y' + w'z' + xz'$

A prime implicant is a product term obtained by combining the maximum possible number of adjacent squares in the map.

The prime implicants of a function can be obtained from the map by combining all possible maximum numbers of squares.



**FIGURE 3.11**

Simplification using prime implicants

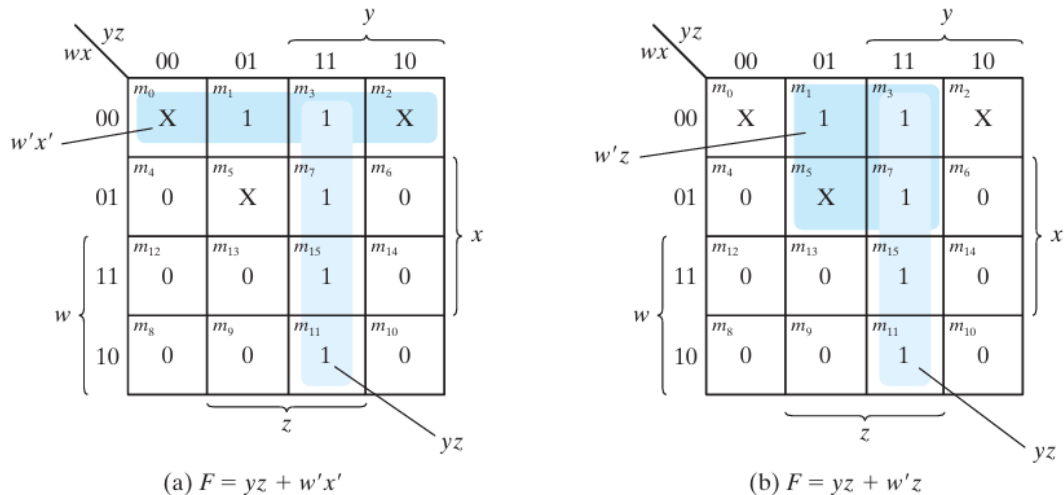
Five-Variable Map Maps for more than four variables are not as simple to use as maps for four or fewer variables. A five-variable map needs 32 squares and a six-variable map needs 64 squares. When the number of variables becomes large, the number of squares becomes excessive and the geometry for combining adjacent squares becomes more involved. Maps for more than four variables are difficult to use and will not be considered here.

#### Don't Care Conditions:

The logical sum of the minterms associated with a Boolean function specifies the conditions under which the function is equal to 1. The function is equal to 0 for the rest of the minterms. This pair of conditions assumes that all the combinations of the values for the variables of the function are valid. In practice, in some applications the function is not specified for certain combinations of the variables. As an example, the four-bit binary code for the decimal digits has six combinations that are not used and consequently are considered to be unspecified. Functions that have unspecified outputs for some input combinations are called incompletely specified functions. In most applications, we simply don't care what value is assumed by the function for the unspecified minterms. For this reason, it is customary to call the unspecified minterms of a function don't-care conditions. These don't-care conditions can be used on a map to provide further simplification of the Boolean expression. A don't-care minterm is a combination of variables whose logical value is not specified. Such a minterm cannot be marked with a 1 in the map, because it would require that the function always be a 1 for such a combination. Likewise, putting a 0 on the square requires the function to be 0. To distinguish the don't-care condition from 1's and 0's, an X is used. Thus, an X inside a square in the map indicates that we don't care whether the value of 0 or 1 is assigned to F for the particular minterm. In choosing adjacent squares to simplify the function in a map, the don't-care minterms may be assumed to be either 0 or 1. When simplifying the function, we can choose to include each don't-care minterm with either the 1's or the 0's, depending on which combination gives the simplest expression.

$$F(w, x, y, z) = (1, 3, 7, 11, 15)$$

$$d(w, x, y, z) = (0, 2, 5)$$

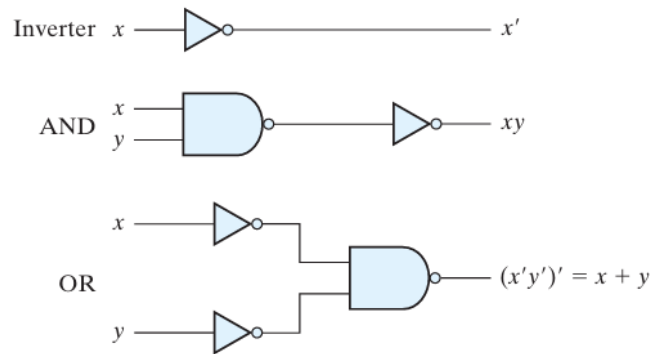


**FIGURE 3.15**  
Example with don't-care conditions

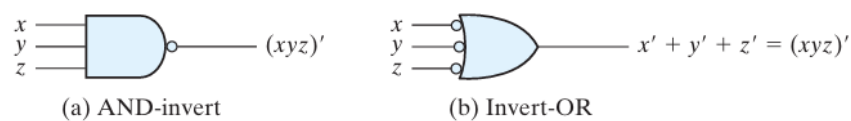
NAND and NOR implementation:

Digital circuits are frequently constructed with NAND or NOR gates rather than with AND and OR gates. NAND and NOR gates are easier to fabricate with electronic components and are the basic gates used in all IC digital logic families. Because of the prominence of NAND and NOR gates in the design of digital circuits, rules and procedures have been developed for the conversion from Boolean functions given in terms of AND, OR, and NOT into equivalent NAND and NOR logic diagrams.

**NAND Circuits** The NAND gate is said to be a universal gate because any logic circuit can be implemented with it. To show that any Boolean function can be implemented with NAND gates, we need only show that the logical operations of AND, OR, and complement can be obtained with NAND gates alone. This is indeed shown in Fig. 3.16. The complement operation is obtained from a one-input NAND gate that behaves exactly like an inverter. The AND operation requires two NAND gates. The first produces the NAND operation and the second inverts the logical sense of the signal. The OR operation is achieved through a NAND gate with additional inverters in each input. A convenient way to implement a Boolean function with NAND gates is to obtain the simplified Boolean function in terms of Boolean operators and then convert the function to NAND logic. The conversion of an algebraic expression from AND, OR, and complement to NAND can be done by simple circuit manipulation techniques that change AND-OR diagrams to NAND diagrams.



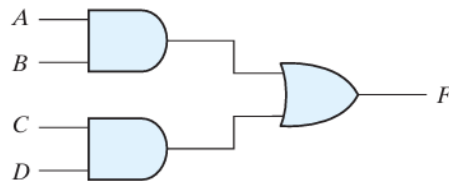
**FIGURE 3.16**  
Logic operations with NAND gates



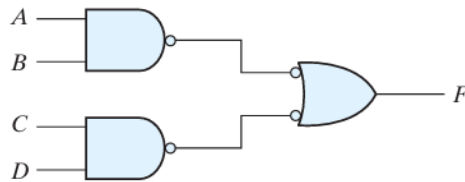
**FIGURE 3.17**  
Two graphic symbols for a three-input NAND gate

## Two-Level Implementation

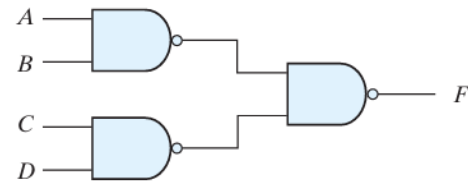
The implementation of Boolean functions with NAND gates requires that the functions be in sum-of-products form. To see the relationship between a sum-of-products expression and its equivalent NAND implementation, consider the logic diagrams drawn in Fig. 3.18. All three diagrams are equivalent and implement the function  $F = AB + CD$ . The function is implemented in Fig. 3.18(a) with AND and OR gates. In Fig. 3.18(b), the AND gates are replaced by NAND gates and the OR gate is replaced by a NAND gate with an OR-invert graphic symbol. Remember that a bubble denotes complementation and two bubbles along the same line represent double complementation, so both can be removed. Removing the bubbles on the gates of (b) produces the circuit of (a). Therefore, the two diagrams implement the same function and are equivalent.



(a)



(b)

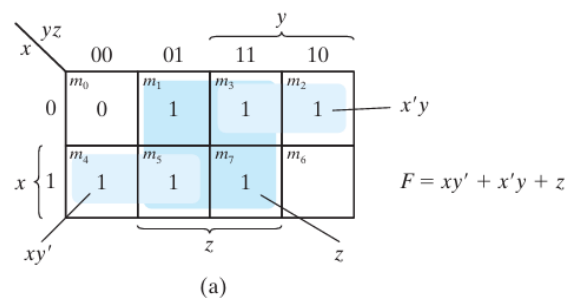


(c)

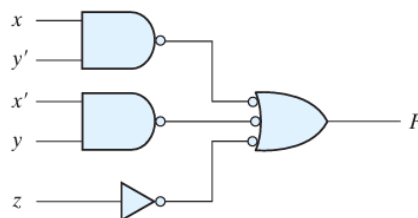
**FIGURE 3.18**

Three ways to implement  $F = AB + CD$

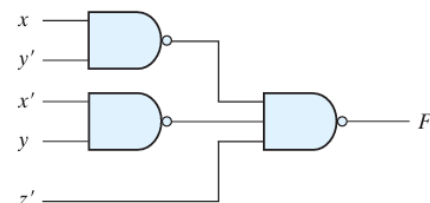
Implement the following Boolean function with NAND gates:  $F(x, y, z) = (1, 2, 3, 4, 5, 7)$



(a)



(b)



(c)

**FIGURE 3.19**

Solution to Example 3.9

The procedure described in the previous example indicates that a Boolean function can be implemented with two levels of NAND gates. The procedure for obtaining the logic diagram from a Boolean function is as follows:

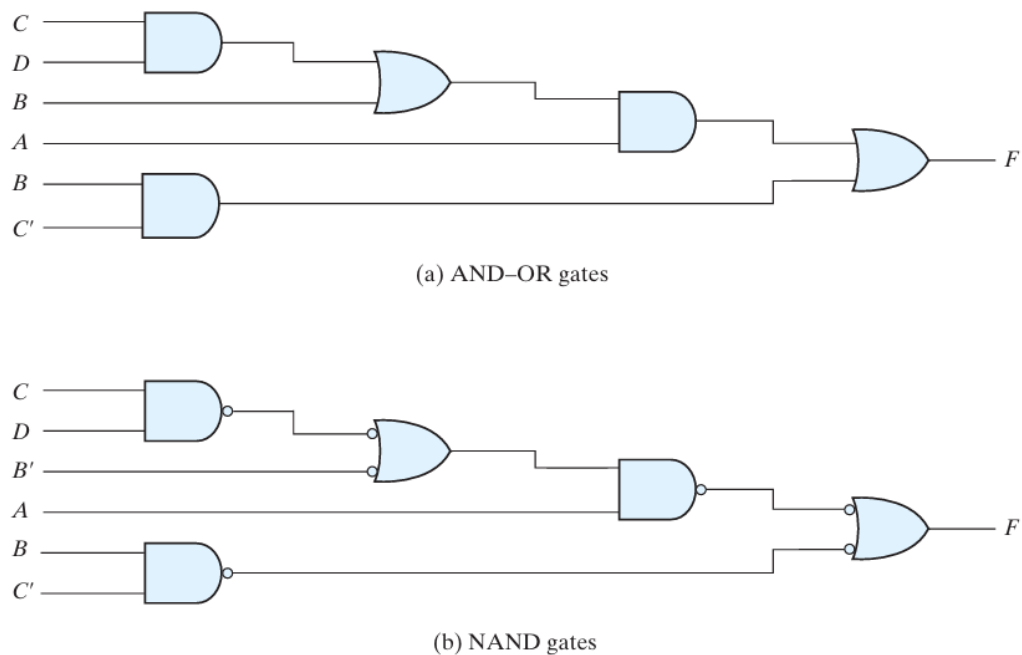
1. Simplify the function and express it in sum-of-products form.
2. Draw a NAND gate for each product term of the expression that has at least two literals. The inputs to each NAND gate are the literals of the term. This procedure produces a group of first-level gates.
3. Draw a single gate using the AND-invert or the invert-OR graphic symbol in the second level, with inputs coming from outputs of first-level gates.



4. A term with a single literal requires an inverter in the first level. However, if the single literal is complemented, it can be connected directly to an input of the second level NAND gate.

### Multilevel NAND Circuits

The standard form of expressing Boolean functions results in a two-level implementation. There are occasions, however, when the design of digital systems results in gating structures with three or more levels. The most common procedure in the design of multilevel circuits is to express the Boolean function in terms of AND, OR, and complement operations. The function can then be implemented with AND and OR gates. After that, if necessary, it can be converted into an all-NAND circuit.



**FIGURE 3.20**  
Implementing  $F = A(CD + B) + BC'$

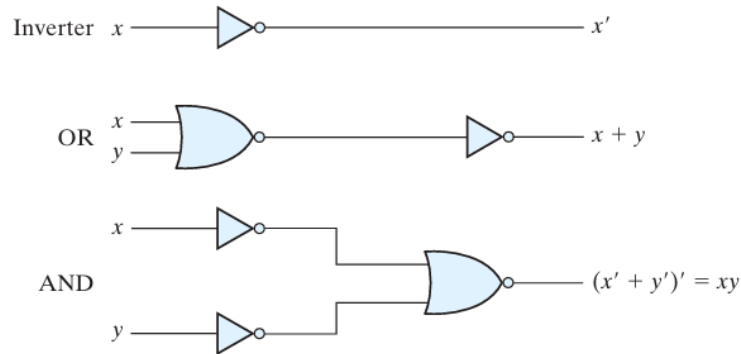
The general procedure for converting a multilevel AND-OR diagram into an all-NAND diagram using mixed notation is as follows:

1. Convert all AND gates to NAND gates with AND-invert graphic symbols.
2. Convert all OR gates to NAND gates with invert-OR graphic symbols.
3. Check all the bubbles in the diagram. For every bubble that is not compensated by another small circle along the same line, insert an inverter (a one-input NAND gate) or complement the input literal.

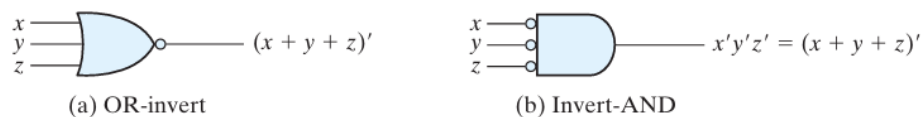
### NOR Implementation

The NOR operation is the dual of the NAND operation. Therefore, all procedures and rules for NOR logic are the duals of the corresponding procedures and rules developed for NAND logic. The NOR gate is another universal gate that can be used to implement any Boolean function. The implementation of the complement, OR, and AND operations with NOR gates is shown in Fig. 3.22. The complement operation is obtained from a one input NOR gate that behaves

exactly like an inverter. The OR operation requires two NOR gates, and the AND operation is obtained with a NOR gate that has inverters in each input. The two graphic symbols for the mixed notation are shown in Fig. 3.23 . The OR-invert symbol defines the NOR operation as an OR followed by a complement. The invert-AND symbol complements each input and then performs an AND operation. The two symbols designate the same NOR operation and are logically identical because of DeMorgan's theorem.



**FIGURE 3.22**  
Logic operations with NOR gates

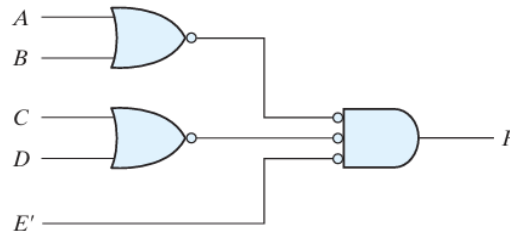


**FIGURE 3.23**  
Two graphic symbols for the NOR gate

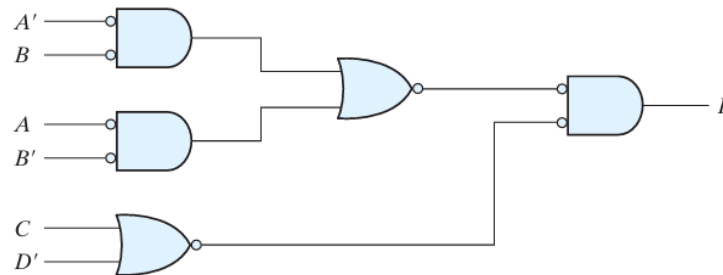
A two-level implementation with NOR gates requires that the function be simplified into product-of-sums form. Remember that the simplified product-of-sums expression is obtained from the map by combining the 0's and complementing. A product-of-sums expression is implemented with a first level of OR gates that produce the sum terms followed by a second-level AND gate to produce the product. The transformation from the OR-AND diagram to a NOR diagram is achieved by changing the OR gates to NOR gates with OR-invert graphic symbols and the AND gate to a NOR gate with an invert-AND graphic symbol. A single literal term going into the second-level gate must be complemented.

NOR implementation of a function expressed as a product of sums:  $F = (A + B)(C + D)E$

The transformation of the AND-OR diagram of Fig. 3.21 (a) into a NOR diagram is shown in Fig. 3.25 . The Boolean function for this circuit is  $F = (AB + AB)(C + D)$



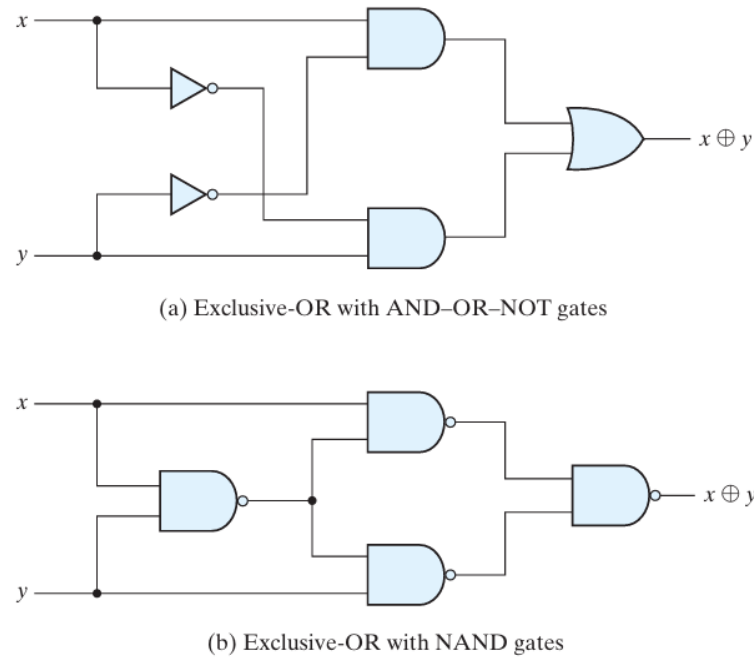
**FIGURE 3.24**  
Implementing  $F = (A + B)(C + D)E$



**FIGURE 3.25**  
Implementing  $F = (AB' + A'B)(C + D')$  with NOR gates

## Exclusive OR

The exclusive-OR (XOR), denoted by the symbol  $\{$ , is a logical operation that performs the following Boolean operation:  $x \{ y = xy + xy$ . The exclusive-OR is equal to 1 if only  $x$  is equal to 1 or if only  $y$  is equal to 1 (i.e.,  $x$  and  $y$  differ in value), but not when both are equal to 1 or when both are equal to 0. The exclusive NOR, also known as equivalence, performs the following Boolean operation:  $(x \{ y) = xy + xy$ . The exclusive-NOR is equal to 1 if both  $x$  and  $y$  are equal to 1 or if both are equal to 0. The exclusive-NOR can be shown to be the complement of the exclusive-OR by means of a truth table or by algebraic manipulation:  $(x \{ y) = (xy + xy) = (x + y)(x + y) = xy + xy$ . The following identities apply to the exclusive-OR operation:  $x \{ 0 = x$ ,  $x \{ 1 = x$ ,  $x \{ x = 0$ ,  $x \{ x = 1$ ,  $x \{ y = x \{ y = (x \{ y)$ . Any of these identities can be proven with a truth table or by replacing the  $\{$  operation by its equivalent Boolean expression. Also, it can be shown that the exclusive-OR operation is both commutative and associative; that is,  $A \{ B = B \{ A$  and  $(A \{ B) \{ C = A \{ (B \{ C) = A \{ B \{ C$ . This means that the two inputs to an exclusive-OR gate can be interchanged without affecting the operation. It also means that we can evaluate a three-variable exclusive-OR operation in any order, and for this reason, three or more variables can be expressed without parentheses. This would imply the possibility of using exclusive-OR gates with three or more inputs. However, multiple-input exclusive-OR gates are difficult to fabricate with hardware. In fact, even a two-input function is usually constructed with other types of gates. A two-input exclusive-OR function is constructed with conventional gates using two inverters, two AND gates, and an OR gate, as shown in Fig. 3.30 (a). Figure 3.30 (b) shows the implementation of the exclusive-OR with four NAND gates. The first NAND gate performs the operation  $(xy) = (x + y)$ .

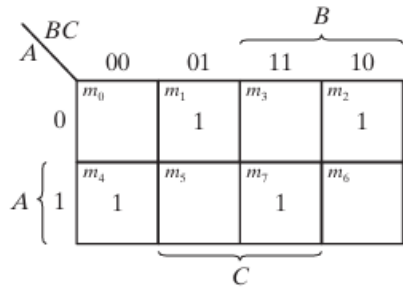


**FIGURE 3.30**  
Exclusive-OR implementations

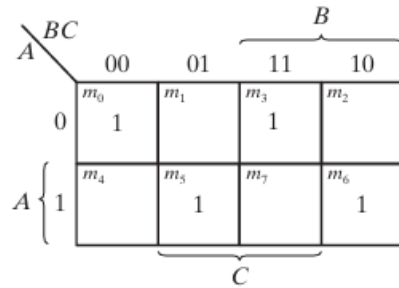
**Odd Function** The exclusive-OR operation with three or more variables can be converted into an ordinary Boolean function by replacing the  $\{$  symbol with its equivalent Boolean expression.

The Boolean expression clearly indicates that the three-variable exclusive-OR function is equal to 1 if only one variable is equal to 1 or if all three variables are equal to 1. Contrary to the two-variable case, in which only one variable must be equal to 1, in the case of three or more variables the requirement is that an odd number of variables be equal to 1. As a consequence, the multiple-variable exclusive-OR operation is defined as an odd function. The Boolean function derived from the three-variable exclusive-OR operation is expressed as the logical sum of four minterms whose binary numerical values are 001, 010, 100, and 111. Each of these binary numbers has an odd number of 1's. The remaining four minterms not included in the function are 000, 011, 101, and 110, and they have an even number of 1's in their binary numerical values. In general, an  $n$ -variable exclusive-OR function is an odd function defined as the logical sum of the  $2^{n-1}$  minterms whose binary numerical values have an odd number of 1's. The definition of an odd function can be clarified by plotting it in a map. Figure 3.31 (a) shows the map for the three-variable exclusive-OR function. The four minterms of the function are a unit distance apart from each other. The odd function is identified from the four minterms whose binary values have an odd number of 1's. The complement of an odd function is an even function. As shown in Fig. 3.31 (b), the three-variable even function is equal to 1 when an even number of its variables is equal to 1 (including the condition that none of the variables is equal to 1). The three-input odd function is implemented by means of two-input exclusive-OR gates, as shown in Fig. 3.32 (a). The complement of an odd function is obtained by replacing the output gate with an exclusive-NOR gate, as shown in Fig. 3.32 (b).

$$\begin{aligned}
 A \oplus B \oplus C \oplus D &= (AB' + A'B) \oplus (CD' + C'D) \\
 &= (AB' + A'B)(CD + C'D') + (AB + A'B')(CD' + C'D) \\
 &= \Sigma(1, 2, 4, 7, 8, 11, 13, 14)
 \end{aligned}$$



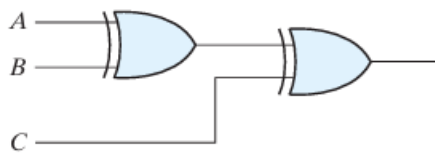
(a) Odd function  $F = A \oplus B \oplus C$



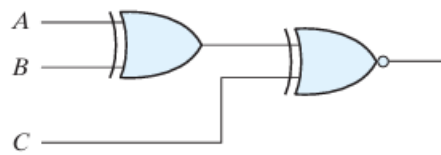
(b) Even function  $F = (A \oplus B \oplus C)'$

**FIGURE 3.31**

Map for a three-variable exclusive-OR function



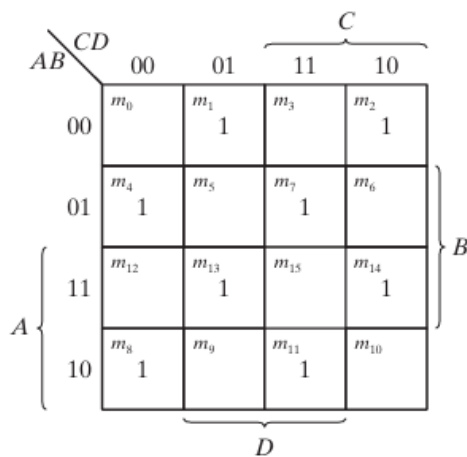
(a) 3-input odd function



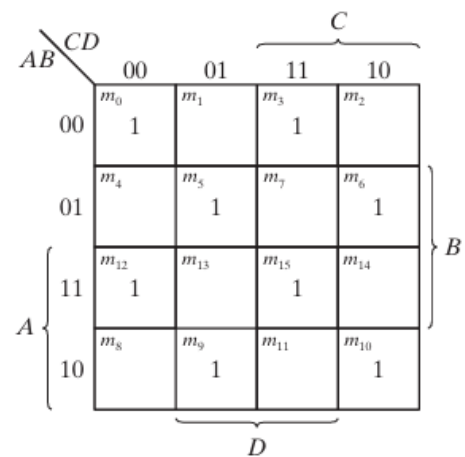
(b) 3-input even function

**FIGURE 3.32**

Logic diagram of odd and even functions



(a) Odd function  $F = A \oplus B \oplus C \oplus D$



(b) Even function  $F = (A \oplus B \oplus C \oplus D)'$

**FIGURE 3.33**

Map for a four-variable exclusive-OR function

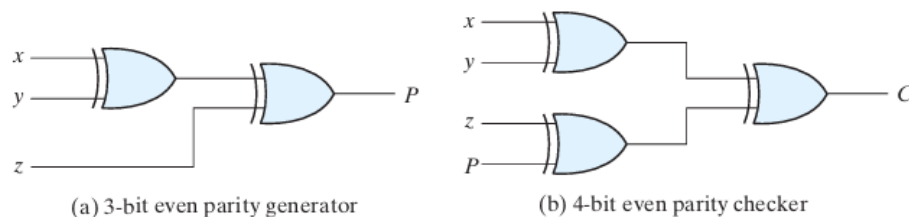
There are 16 minterms for a four-variable Boolean function. Half of the minterms have binary numerical values with an odd number of 1's; the other half of the minterms have binary numerical values with an even number of 1's. In plotting the function in the map, the binary numerical value for a minterm is determined from the row and column numbers of the square that represents the minterm. The map of Fig. 3.33 (a) is a plot of the four-variable exclusive-

OR function. This is an odd function because the binary values of all the minterms have an odd number of 1's. The complement of an odd function is an even function. As shown in Fig. 3.33 (b), the four-variable even function is equal to 1 when an even number of its variables is equal to 1.

Parity Generation and Checking Exclusive-OR functions are very useful in systems requiring error detection and correction codes. As discussed in Section 1.6, a parity bit is used for the purpose of detecting errors during the transmission of binary information. A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even. The message, including the parity bit, is transmitted and then checked at the receiving end for errors. An error is detected if the checked parity does not correspond with the one transmitted. The circuit that generates the parity bit in the transmitter is called a parity generator. The circuit that checks the parity in the receiver is called a parity checker.

**Table 3.3**  
*Even-Parity-Generator Truth Table*

| Three-Bit Message |          |          | Parity Bit |
|-------------------|----------|----------|------------|
| <i>x</i>          | <i>y</i> | <i>z</i> | <i>P</i>   |
| 0                 | 0        | 0        | 0          |
| 0                 | 0        | 1        | 1          |
| 0                 | 1        | 0        | 1          |
| 0                 | 1        | 1        | 0          |
| 1                 | 0        | 0        | 1          |
| 1                 | 0        | 1        | 0          |
| 1                 | 1        | 0        | 0          |
| 1                 | 1        | 1        | 1          |



**FIGURE 3.34**  
Logic diagram of a parity generator and checker

**Table 3.4**  
*Even-Parity-Checker Truth Table*

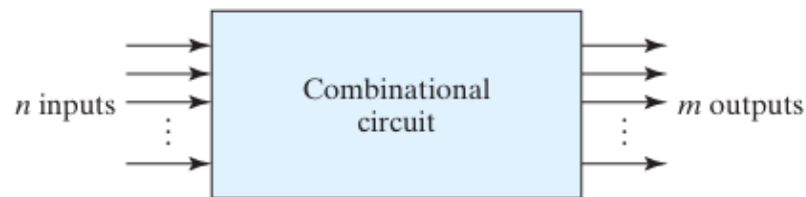
| Four Bits Received |          |          |          | Parity Error Check |
|--------------------|----------|----------|----------|--------------------|
| <i>x</i>           | <i>y</i> | <i>z</i> | <i>P</i> | <i>C</i>           |
| 0                  | 0        | 0        | 0        | 0                  |
| 0                  | 0        | 0        | 1        | 1                  |
| 0                  | 0        | 1        | 0        | 1                  |
| 0                  | 0        | 1        | 1        | 0                  |
| 0                  | 1        | 0        | 0        | 1                  |
| 0                  | 1        | 0        | 1        | 0                  |
| 0                  | 1        | 1        | 0        | 0                  |
| 0                  | 1        | 1        | 1        | 1                  |
| 1                  | 0        | 0        | 0        | 1                  |
| 1                  | 0        | 0        | 1        | 0                  |
| 1                  | 0        | 1        | 0        | 0                  |
| 1                  | 0        | 1        | 1        | 1                  |
| 1                  | 1        | 0        | 0        | 0                  |
| 1                  | 1        | 0        | 1        | 1                  |
| 1                  | 1        | 1        | 0        | 1                  |
| 1                  | 1        | 1        | 1        | 0                  |

The logic diagram of the parity checker is shown in Fig. 3.34 (b). It is worth noting that the parity generator can be implemented with the circuit of Fig. 3.34 (b) if the input *P* is connected to logic 0 and the output is marked with *P*. This is because  $z \oplus 0 = z$ , causing the value of *z* to pass through the gate unchanged. The advantage of this strategy is that the same circuit can be used for both parity generation and checking. It is obvious from the foregoing example that parity generation and checking circuits always have an output function that includes half of the minterms whose numerical values have either an odd or even number of 1's. As a consequence, they can be implemented with exclusive-OR gates. A function with an even number of 1's is the complement of an odd function. It is implemented with exclusive-OR gates, except that the gate associated with the output must be an exclusive-NOR to provide the required complementation.

## COMBINATIONAL LOGIC:

Logic circuits for digital systems may be combinational or sequential. A combinational circuit consists of logic gates whose outputs at any time are determined from only the present combination of inputs. A combinational circuit performs an operation that can be specified logically by a set of Boolean functions. In contrast, sequential circuits employ storage elements in addition to logic gates. Their outputs are a function of the inputs and the state of the storage elements. Because the state of the storage elements is a function of previous inputs, the outputs of a sequential circuit depend not only on present values of inputs, but also on past inputs, and the circuit behavior must be specified by a time sequence of inputs and internal states.

A combinational circuit consists of an interconnection of logic gates. Combinational logic gates react to the values of the signals at their inputs and produce the value of the output signal, transforming binary information from the given input data to a required output data. A block diagram of a combinational circuit is shown in Fig. 4.1. The  $n$  input binary variables come from an external source; the  $m$  output variables are produced by the internal combinational logic circuit and go to an external destination. Each input and output variable exists physically as an analog signal whose values are interpreted to be a binary signal that represents logic 1 and logic 0. (Note: Logic simulators show only 0's and 1's, not the actual analog signals.)



**FIGURE 4.1**

**Block diagram of combinational circuit**

If the registers are included with the combinational gates, then the total circuit must be considered to be a sequential circuit. For  $n$  input variables, there are  $2^n$  possible combinations of the binary inputs. For each possible input combination, there is one possible value for each output variable. Thus, a combinational circuit can be specified with a truth table that lists the output values for each combination of input variables. A combinational circuit also can be described by  $m$  Boolean functions, one for each output variable. Each output function is expressed in terms of the  $n$  input variables. In Chapter 1, we learned about binary numbers and binary codes that represent discrete quantities of information. The binary variables are represented physically by electric voltages or some other type of signal. The signals can be manipulated in digital logic gates to perform required functions. In Chapter 2, we introduced Boolean algebra as a way to express logic functions algebraically. In Chapter 3, we learned how to simplify Boolean functions to achieve economical (simpler) gate implementations. The purpose of the current chapter is to use the knowledge acquired in previous chapters to formulate systematic analysis and design procedures for combinational circuits. The solution of some typical examples will provide a useful catalog of elementary functions that are important for the understanding of digital systems. We'll address three tasks: (1) Analyze the behavior of a given logic circuit, (2) synthesize a circuit that will have a given behavior, and (3) write hardware description language (HDL) models for some common circuits.



## Analysis Procedure:

The analysis of a combinational circuit requires that we determine the function that the circuit implements. This task starts with a given logic diagram and culminates with a set of Boolean functions, a truth table, or, possibly, an explanation of the circuit operation.

If the logic diagram to be analyzed is accompanied by a function name or an explanation of what it is assumed to accomplish, then the analysis problem reduces to a verification of the stated function. The analysis can be performed manually by finding the Boolean functions or truth table or by using a computer simulation program. The first step in the analysis is to make sure that the given circuit is combinational and not sequential. The diagram of a combinational circuit has logic gates with no feedback paths or memory elements. A feedback path is a connection from the output of one gate to the input of a second gate whose output forms part of the input to the first gate. Feedback paths in a digital circuit define a sequential circuit and must be analyzed by special methods and will not be considered here. Once the logic diagram is verified to be that of a combinational circuit, one can proceed to obtain the output Boolean functions or the truth table. If the function of the circuit is under investigation, then it is necessary to interpret the operation of the circuit from the derived Boolean functions or truth table. The success of such an investigation is enhanced if one has previous experience and familiarity with a wide variety of digital circuits. To obtain the output Boolean functions from a logic diagram, we proceed as follows: 1. Label all gate outputs that are a function of input variables with arbitrary symbols— but with meaningful names. Determine the Boolean functions for each gate output. 2. Label the gates that are a function of input variables and previously labeled gates with other arbitrary symbols. Find the Boolean functions for these gates. 3. Repeat the process outlined in step 2 until the outputs of the circuit are obtained. 4. By repeated substitution of previously defined functions, obtain the output Boolean functions in terms of input variables.

$$F_2 = AB + AC + BC$$

$$T_1 = A + B + C$$

$$T_2 = ABC$$

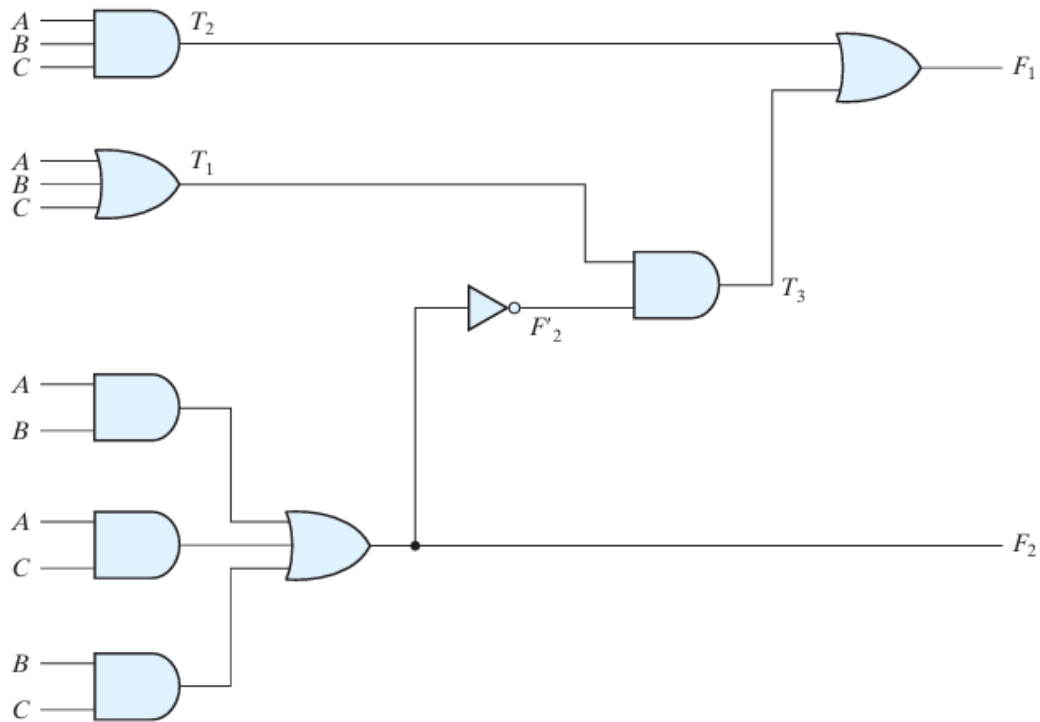
Next, we consider outputs of gates that are a function of already defined symbols:

$$T_3 = F_2 T_1$$

$$F_1 = T_3 + T_2$$

To obtain  $F_1$  as a function of  $A$ ,  $B$ , and  $C$ , we form a series of substitutions as follows:

$$\begin{aligned} F_1 &= T_3 + T_2 = F_2 T_1 + ABC = (AB + AC + BC)'(A + B + C) + ABC \\ &= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC \\ &= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC \\ &= A'BC' + A'B'C + AB'C' + ABC \end{aligned}$$



**FIGURE 4.2**  
Logic diagram for analysis example

The derivation of the truth table for a circuit is a straightforward process once the output Boolean functions are known. To obtain the truth table directly from the logic diagram without going through the derivations of the Boolean functions, we proceed as follows: 1. Determine the number of input variables in the circuit. For  $n$  inputs, form the  $2^n$  possible input combinations and list the binary numbers from 0 to  $(2^n - 1)$  in a table. 2. Label the outputs of selected gates with arbitrary symbols. 3. Obtain the truth table for the outputs of those gates which are a function of the input variables only. 4. Proceed to obtain the truth table for the outputs of those gates which are a function of previously defined values until the columns for all outputs are determined.

**Table 4.1**  
*Truth Table for the Logic Diagram of Fig. 4.2*

| <b>A</b> | <b>B</b> | <b>C</b> | <b><math>F_2</math></b> | <b><math>F'_2</math></b> | <b><math>T_1</math></b> | <b><math>T_2</math></b> | <b><math>T_3</math></b> | <b><math>F_1</math></b> |
|----------|----------|----------|-------------------------|--------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
| 0        | 0        | 0        | 0                       | 1                        | 0                       | 0                       | 0                       | 0                       |
| 0        | 0        | 1        | 0                       | 1                        | 1                       | 0                       | 1                       | 1                       |
| 0        | 1        | 0        | 0                       | 1                        | 1                       | 0                       | 1                       | 1                       |
| 0        | 1        | 1        | 1                       | 0                        | 1                       | 0                       | 0                       | 0                       |
| 1        | 0        | 0        | 0                       | 1                        | 1                       | 0                       | 1                       | 1                       |
| 1        | 0        | 1        | 1                       | 0                        | 1                       | 0                       | 0                       | 0                       |
| 1        | 1        | 0        | 1                       | 0                        | 1                       | 0                       | 0                       | 0                       |
| 1        | 1        | 1        | 1                       | 0                        | 1                       | 1                       | 0                       | 1                       |

### Design Procedure:

The design of combinational circuits starts from the specification of the design objective and culminates in a logic circuit diagram or a set of Boolean functions from which the logic diagram can be obtained. The procedure involves the following steps: 1. From the specifications of the circuit, determine the required number of inputs and outputs and assign a symbol to each. 2. Derive the truth table that defines the required relationship between inputs and outputs. 3. Obtain the simplified Boolean functions for each output as a function of the input variables. 4. Draw the logic diagram and verify the correctness of the design (manually or by simulation).

A truth table for a combinational circuit consists of input columns and output columns. The input columns are obtained from the  $2^n$  binary numbers for the  $n$  input variables. The binary values for the outputs are determined from the stated specifications. The output functions specified in the truth table give the exact definition of the combinational circuit. It is important that the verbal specifications be interpreted correctly in the truth table, as they are often incomplete, and any wrong interpretation may result in an incorrect truth table. The output binary functions listed in the truth table are simplified by any available method, such as algebraic manipulation, the map method, or a computer-based simplification program. Frequently, there is a variety of simplified expressions from which to choose. In a particular application, certain criteria will serve as a guide in the process of choosing an implementation. A practical design must consider such constraints as the number of gates, number of inputs to a gate, propagation time of the signal through the gates, number of interconnections, limitations of the driving capability of each gate (i.e., the number of gates to which the output of the circuit may be connected), and various other criteria that must be taken into consideration when designing integrated circuits. Since the importance of each constraint is dictated by the particular application, it is difficult to make a general statement about what constitutes an acceptable implementation. In most cases, the simplification begins by satisfying an elementary objective, such as producing the simplified Boolean functions in a standard form.

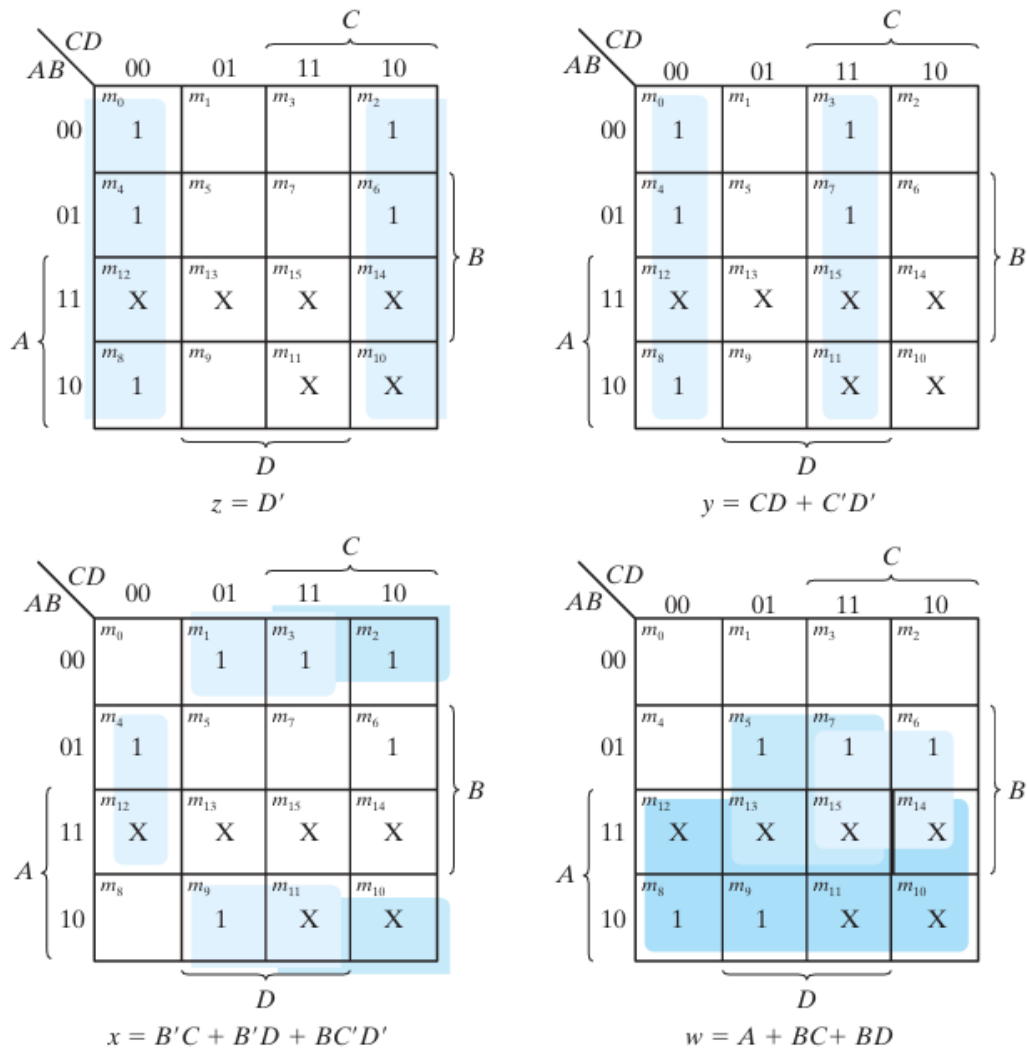
### Code Conversion Example

The availability of a large variety of codes for the same discrete elements of information results in the use of different codes by different digital systems. It is sometimes necessary to use the output of one system as the input to another. A conversion circuit must be inserted between the two systems if each uses different codes for the same information. Thus, a code converter is a circuit that makes the two systems compatible even though each uses a different binary code. To convert from binary code A to binary code B, the input lines must supply the bit combination of elements as specified by code A and the output lines must generate the corresponding bit combination of code B. A combinational circuit performs this transformation by means of logic gates. The design procedure will be illustrated by an example that converts binary coded decimal (BCD) to the excess-3 code for the decimal digits.

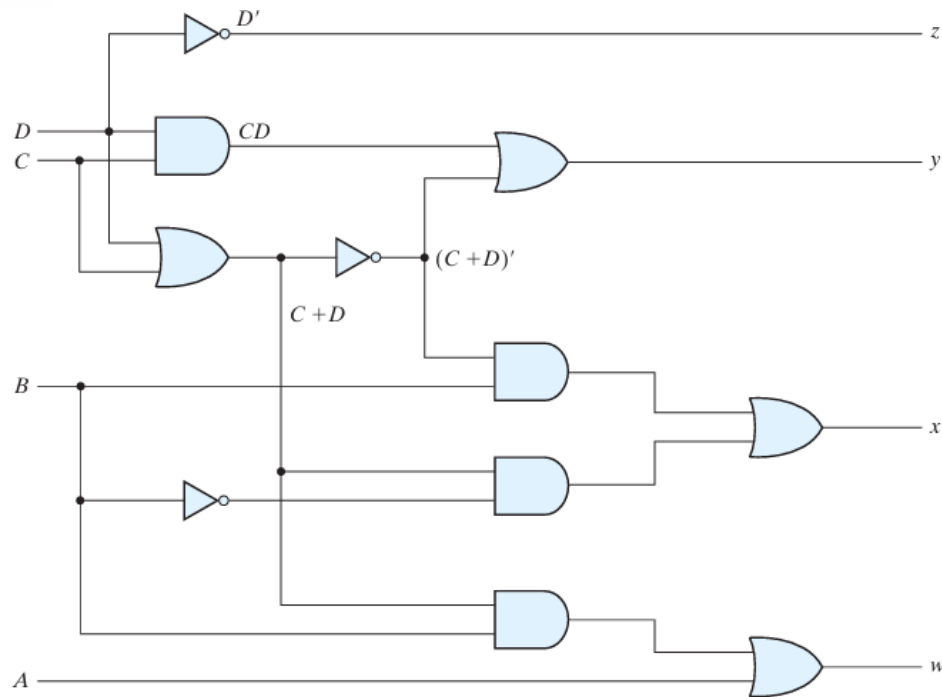
**Table 4.2**  
*Truth Table for Code Conversion Example*

| Input BCD |          |          |          | Output Excess-3 Code |          |          |          |
|-----------|----------|----------|----------|----------------------|----------|----------|----------|
| <i>A</i>  | <i>B</i> | <i>C</i> | <i>D</i> | <i>w</i>             | <i>x</i> | <i>y</i> | <i>z</i> |
| 0         | 0        | 0        | 0        | 0                    | 0        | 1        | 1        |
| 0         | 0        | 0        | 1        | 0                    | 1        | 0        | 0        |
| 0         | 0        | 1        | 0        | 0                    | 1        | 0        | 1        |
| 0         | 0        | 1        | 1        | 0                    | 1        | 1        | 0        |
| 0         | 1        | 0        | 0        | 0                    | 1        | 1        | 1        |
| 0         | 1        | 0        | 1        | 1                    | 0        | 0        | 0        |
| 0         | 1        | 1        | 0        | 1                    | 0        | 0        | 1        |
| 0         | 1        | 1        | 1        | 1                    | 0        | 1        | 0        |
| 1         | 0        | 0        | 0        | 1                    | 0        | 1        | 1        |
| 1         | 0        | 0        | 1        | 1                    | 1        | 0        | 0        |

We designate the four input binary variables by the symbols A, B, C, and D, and the four output variables by w, x, y, and z. The truth table relating the input and output variables is shown in Table 4.2. The bit combinations for the inputs and their corresponding outputs are obtained directly from Section 1.7. Note that four binary variables may have 16 bit combinations, but only 10 are listed in the truth table. The six bit combinations not listed for the input variables are don't-care combinations. These values have no meaning in BCD and we assume that they will never occur in actual operation of the circuit. Therefore, we are at liberty to assign to the output variables either a 1 or a 0, whichever gives a simpler circuit. A two-level logic diagram for each output may be obtained directly from the Boolean expressions derived from the maps. There are various other possibilities for a logic diagram that implements this circuit. The expressions obtained in Fig. 4.3 may be manipulated algebraically for the purpose of using common gates for two or more outputs. This manipulation, shown next, illustrates the flexibility obtained with multiple-output systems



**FIGURE 4.3**  
Maps for BCD-to-excess-3 code converter



**FIGURE 4.4**  
Logic diagram for BCD-to-excess-3 code converter

implementation will require inverters for variables B, C, and D, and the second implementation will require inverters for variables B and D. Thus, the three-level logic circuit requires fewer gates, all of which in turn require no more than two inputs.

#### Binary Adder-subtractor:

Digital computers perform a variety of information-processing tasks. Among the functions encountered are the various arithmetic operations. The most basic arithmetic operation is the addition of two binary digits. This simple addition consists of four possible elementary operations:  $0 + 0 = 0$ ,  $0 + 1 = 1$ ,  $1 + 0 = 1$ , and  $1 + 1 = 10$ . The first three operations produce a sum of one digit, but when both augend and addend bits are equal to 1, the binary sum consists of two digits. The higher significant bit of this result is called a carry. When the augend and addend numbers contain more significant digits, the carry obtained from the addition of two bits is added to the next higher order pair of significant bits. A combinational circuit that performs the addition of two bits is called a half adder. One that performs the addition of three bits (two significant bits and a previous carry) is a full adder. The names of the circuits stem from the fact that two half adders can be employed to implement a full adder. A binary adder-subtractor is a combinational circuit that performs the arithmetic operations of addition and subtraction with binary numbers. We will develop this circuit by means of a hierarchical design. The half adder design is carried out first, from which we develop the full adder. Connecting  $n$  full adders in cascade produces a binary adder for two  $n$ -bit numbers. The subtraction circuit is included in a complementing circuit.

#### Half adder:

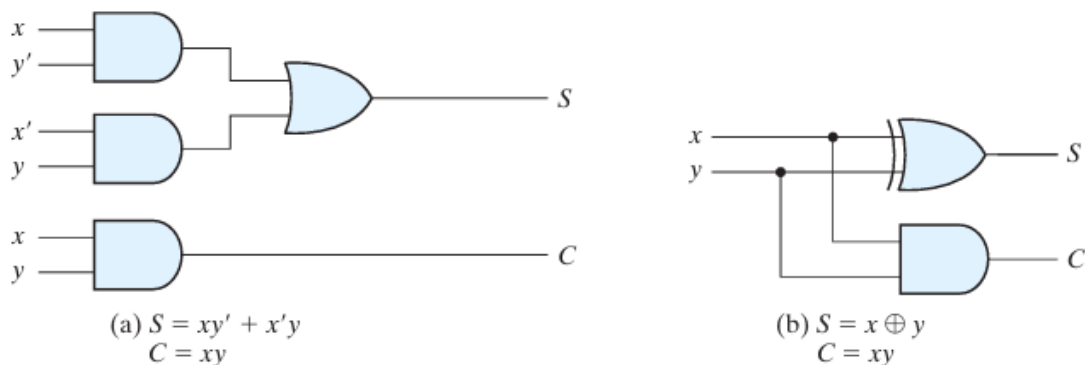
From the verbal explanation of a half adder, we find that this circuit needs two binary inputs and two binary outputs. The input variables designate the augend and addend bits; the output variables produce the sum and carry. We assign symbols  $x$  and  $y$  to the two inputs and  $S$  (for sum) and  $C$  (for carry) to the outputs. The truth table for the half adder is listed in Table 4.3 . The  $C$  output is 1 only when both inputs are 1. The  $S$  output represents the least significant bit of the sum. The simplified Boolean functions for the two outputs can be obtained directly from the truth table. The simplified sum-of-products expressions are

$$S = x'y + xy'$$

$$C = xy$$

**Table 4.3**  
*Half Adder*

| $x$ | $y$ | $C$ | $S$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 1   | 0   | 1   |
| 1   | 0   | 0   | 1   |
| 1   | 1   | 1   | 0   |



**FIGURE 4.5**  
**Implementation of half adder**

Full adder:

Addition of  $n$ -bit binary numbers requires the use of a full adder, and the process of addition proceeds on a bit-by-bit basis, right to left, beginning with the least significant bit. After the least significant bit, addition at each position adds not only the respective bits of the words, but must also consider a possible carry bit from addition at the previous position. A full adder is a combinational circuit that forms the arithmetic sum of three bits. It consists of three inputs and two outputs. Two of the input variables, denoted by  $x$  and  $y$ , represent the two significant bits to be added. The third input,  $z$ , represents the carry from the previous lower significant position. Two outputs are necessary because the arithmetic sum of three binary digits ranges in value from 0 to 3, and binary representation of 2 or 3 needs two bits. The two outputs are designated by the symbols  $S$  for sum and  $C$  for carry. The binary variable  $S$  gives the value of the least significant bit of the sum. The binary variable  $C$  gives the output carry formed by adding the input carry and the bits of the words. The truth table of the full adder is listed in

Table 4.4 . The eight rows under the input variables designate all possible combinations of the three variables. The output variables are determined from the arithmetic sum of the input bits. When all input bits are 0, the output is 0. The S output is equal to 1 when only one input is equal to 1 or when all three inputs are equal to 1. The C output has a carry of 1 if two or three inputs are equal to 1.

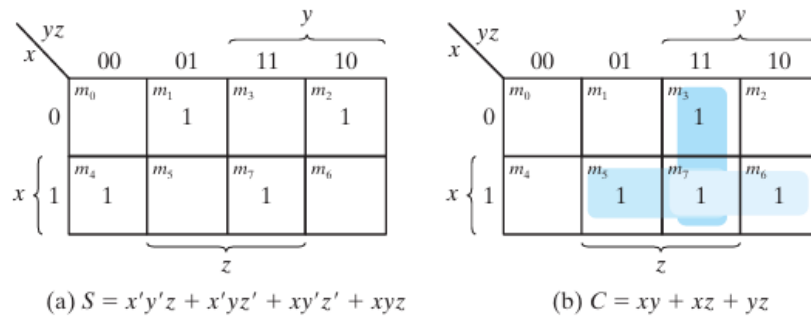
$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$C = xy + xz + yz$$

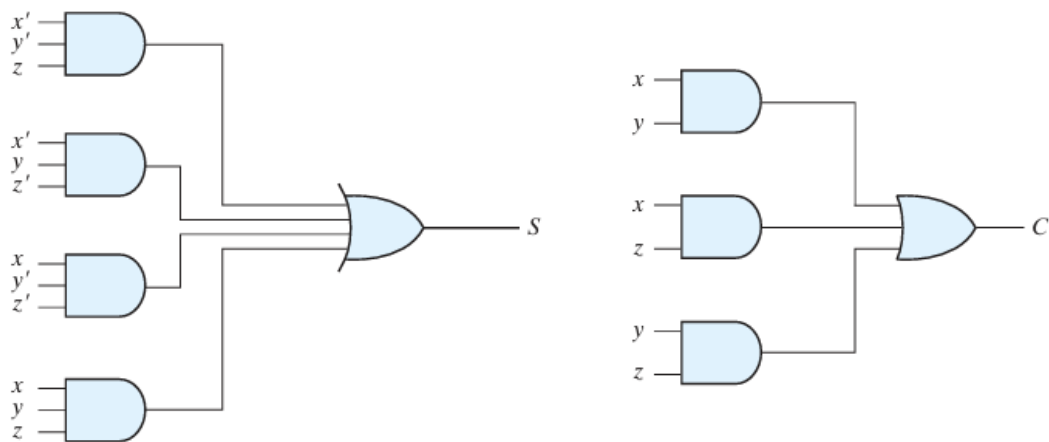
**Table 4.4**  
*Full Adder*

| <b>x</b> | <b>y</b> | <b>z</b> | <b>C</b> | <b>S</b> |
|----------|----------|----------|----------|----------|
| 0        | 0        | 0        | 0        | 0        |
| 0        | 0        | 1        | 0        | 1        |
| 0        | 1        | 0        | 0        | 1        |
| 0        | 1        | 1        | 1        | 0        |
| 1        | 0        | 0        | 0        | 1        |
| 1        | 0        | 1        | 1        | 0        |
| 1        | 1        | 0        | 1        | 0        |
| 1        | 1        | 1        | 1        | 1        |





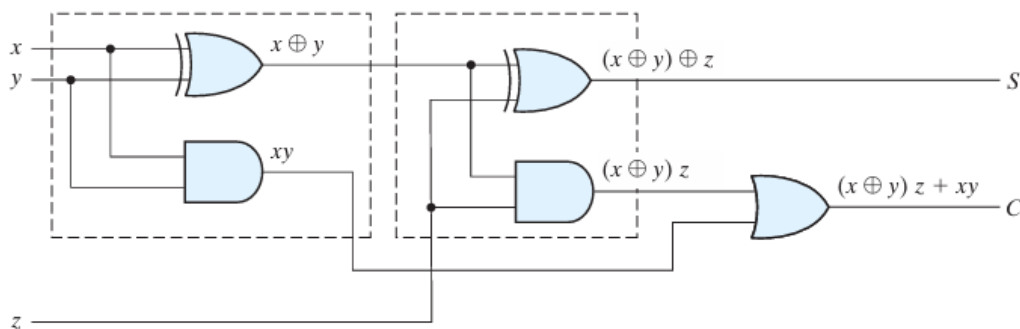
**FIGURE 4.6**  
K-Maps for full adder



**FIGURE 4.7**  
Implementation of full adder in sum-of-products form

Binary adder:

A binary adder is a digital circuit that produces the arithmetic sum of two binary numbers. It can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain.



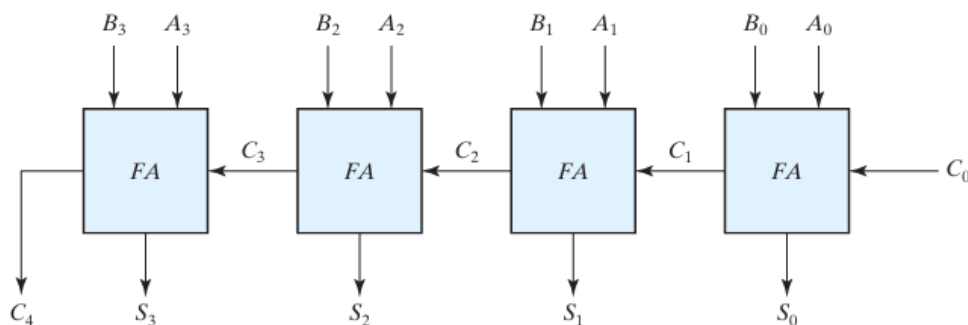
**FIGURE 4.8**  
Implementation of full adder with two half adders and an OR gate

Addition of n-bit numbers requires a chain of n full adders or a chain of one-half adder and n-1 full adders. In the former case, the input carry to the least significant position is fixed at 0. Figure 4.9 shows the interconnection of four full-adder (FA) circuits to provide a four-bit binary

ripple carry adder. The augend bits of A and the addend bits of B are designated by subscript numbers from right to left, with subscript 0 denoting the least significant bit. The carries are connected in a chain through the full adders. The input carry to the adder is  $C_0$ , and it ripples through the full adders to the output carry  $C_4$ . The S outputs generate the required sum bits. An  $n$ -bit adder requires  $n$  full adders, with each output carry connected to the input carry of the next higher order full adder. To demonstrate with a specific example, consider the two binary numbers  $A=1011$  and  $B=0011$ . Their sum  $S=1110$  is formed with the four-bit adder as follows:

| Subscript $i$ : | 3 | 2 | 1 | 0 |           |
|-----------------|---|---|---|---|-----------|
| Input carry     | 0 | 1 | 1 | 0 | $C_i$     |
| Augend          | 1 | 0 | 1 | 1 | $A_i$     |
| Addend          | 0 | 0 | 1 | 1 | $B_i$     |
| Sum             | 1 | 1 | 1 | 0 | $S_i$     |
| Output carry    | 0 | 0 | 1 | 1 | $C_{i+1}$ |

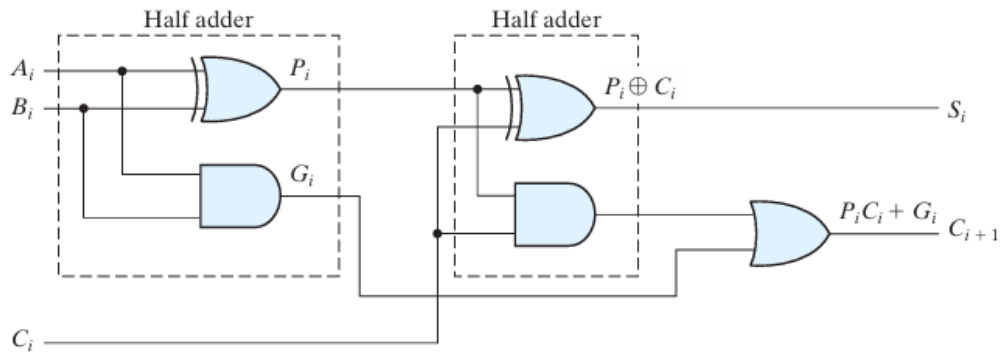
The four-bit adder is a typical example of a standard component. It can be used in many applications involving arithmetic operations.



**FIGURE 4.9**  
Four-bit adder

**Carry Propagation** The addition of two binary numbers in parallel implies that all the bits of the augend and addend are available for computation at the same time. As in any combinational circuit, the signal must propagate through the gates before the correct output sum is available in the output terminals. The total propagation time is equal to the propagation delay of a typical gate, times the number of gate levels in the circuit. The longest propagation delay time in an adder is the time it takes the carry to propagate through the full adders. Since each bit of the sum output depends on the value of the input carry, the value of  $S_i$  at any given stage in the adder will be in its steady-state final value only after the input carry to that stage has been propagated. In this regard, consider output  $S_3$  in Fig. 4.9. Inputs  $A_3$  and  $B_3$  are available as soon as input signals are applied to the adder. However, input carry  $C_3$  does not settle to its final value until  $C_2$  is available from the previous stage. Similarly,  $C_2$  has to wait for  $C_1$  and so on down to  $C_0$ . Thus, only after the carry propagates and ripples through all stages will the last output  $S_3$  and carry  $C_4$  settle to their final correct value. The number of gate levels for the carry propagation can be found from the circuit of the full adder. The circuit is redrawn with

different labels in Fig. 4.10 for convenience. The input and output variables use the subscript  $i$  to denote a typical stage of the adder. The signals at  $P_i$  and  $G_i$  settle to their steady-state values after they propagate through their respective gates. These two signals are common to all half adders and depend on only the input augend and addend bits. The signal from the input carry  $C_i$  to the output carry  $C_{i+1}$  propagates through an AND gate and an OR gate, which constitute two gate levels. If there are four full adders in the adder, the output carry  $C_4$  would have  $2 \times 4 = 8$  gate levels from  $C_0$  to  $C_4$ . For an  $n$ -bit adder, there are  $2n$  gate levels for the carry to propagate from input to output.



**FIGURE 4.10**  
Full adder with  $P$  and  $G$  shown

The carry propagation time is an important attribute of the adder because it limits the speed with which two numbers are added. Although the adder—or, for that matter, any combinational circuit—will always have some value at its output terminals, the outputs will not be correct unless the signals are given enough time to propagate through the gates connected from the inputs to the outputs. Since all other arithmetic operations are implemented by successive additions, the time consumed during the addition process is critical. An obvious solution for reducing the carry propagation delay time is to employ faster gates with reduced delays. However, physical circuits have a limit to their capability. Another solution is to increase the complexity of the equipment in such a way that the carry delay time is reduced. There are several techniques for reducing the carry propagation time in a parallel adder. The most widely used technique employs the principle of carry lookahead logic.

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

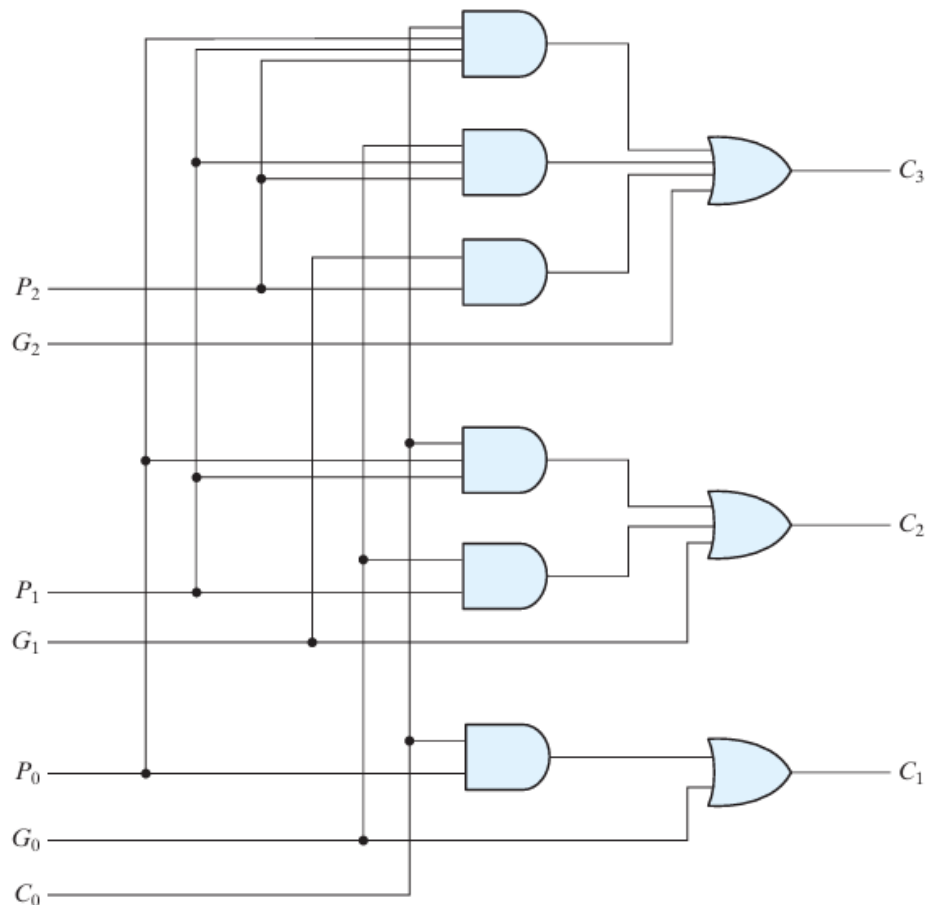
$$C_0 = \text{input carry}$$

$$C_1 = G_0 + P_0 C_0$$

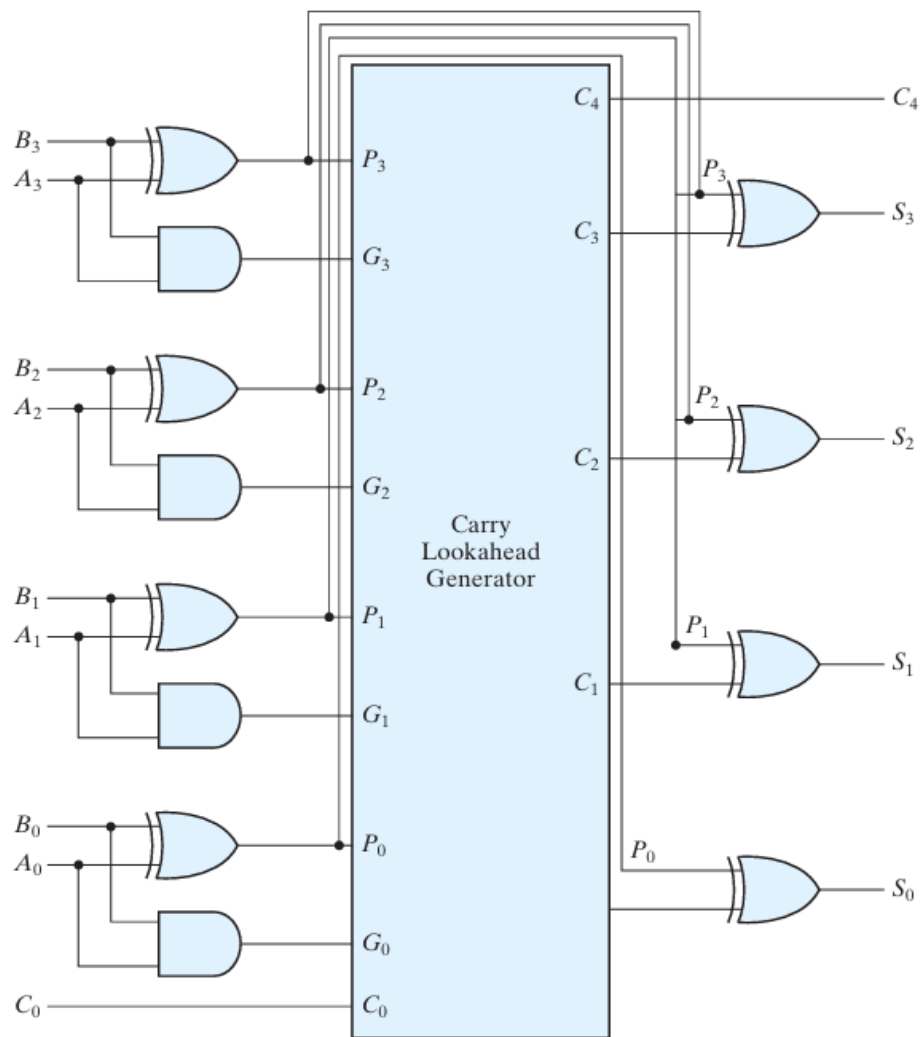
$$C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 = P_2P_1P_0C_0$$

Since the Boolean function for each output carry is expressed in sum-of-products form, each function can be implemented with one level of AND gates followed by an OR gate (or by a two-level NAND). The three Boolean functions for  $C_1$ ,  $C_2$ , and  $C_3$  are implemented in the carry lookahead generator shown in Fig. 4.11. Note that this circuit can add in less time because  $C_3$  does not have to wait for  $C_2$  and  $C_1$  to propagate; in fact,  $C_3$  is propagated at the same time as  $C_1$  and  $C_2$ . This gain in speed of operation is achieved at the expense of additional complexity (hardware). The construction of a four-bit adder with a carry lookahead scheme is shown in Fig. 4.12. Each sum output requires two exclusive-OR gates. The output of the first exclusive-OR gate generates the  $P_i$  variable, and the AND gate generates the  $G_i$  variable. The carries are propagated through the carry lookahead generator (similar to that in Fig. 4.11) and applied as inputs to the second exclusive-OR gate



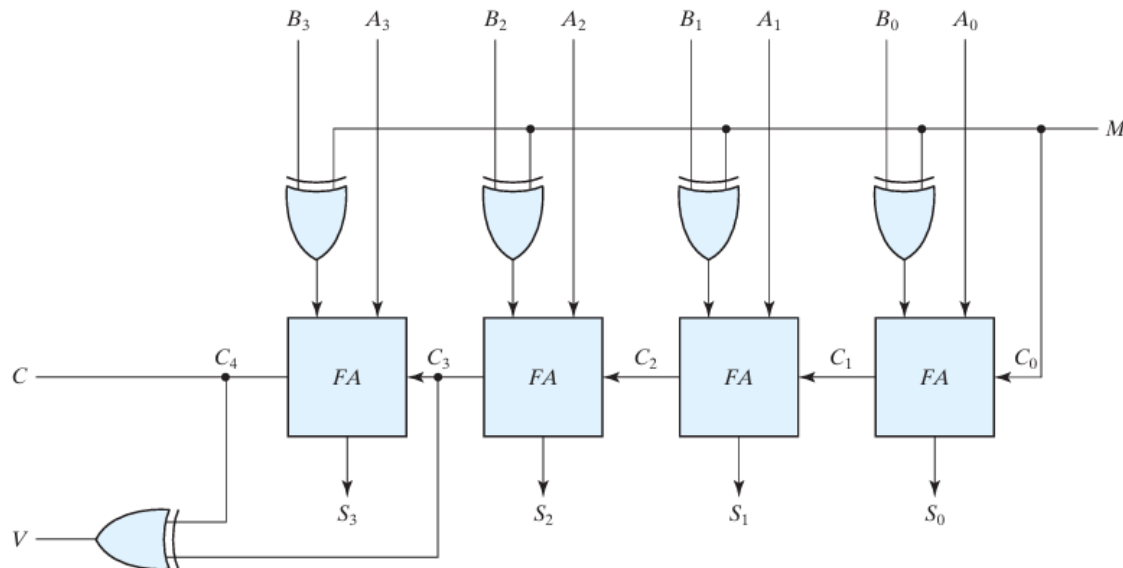
**FIGURE 4.11**  
 Logic diagram of carry lookahead generator



**FIGURE 4.12**  
 Four-bit adder with carry lookahead

### Binary Subtractor

The subtraction of unsigned binary numbers can be done most conveniently by means of complements, as discussed in Section 1.5. Remember that the subtraction  $A - B$  can be done by taking the 2's complement of  $B$  and adding it to  $A$ . The 2's complement can be obtained by taking the 1's complement and adding 1 to the least significant pair of bits. The 1's complement can be implemented with inverters, and a 1 can be added to the sum through the input carry.



**FIGURE 4.13**  
Four-bit adder-subtractor (with overflow detection)

ach data input B and the corresponding input of the full adder. The input carry  $C_0$  must be equal to 1 when subtraction is performed. The operation thus performed becomes  $A$ , plus the 1's complement of  $B$ , plus 1. This is equal to  $A$  plus the 2's complement of  $B$ . For unsigned numbers, that gives  $A - B$  if  $A \geq B$  or the 2's complement of  $1B - A$  if  $A < B$ . For signed numbers, the result is  $A - B$ , provided that there is no overflow. The addition and subtraction operations can be combined into one circuit with one common binary adder by including an exclusive-OR gate with each full adder. A four-bit adder-subtractor circuit is shown in Fig. 4.13. The mode input  $M$  controls the operation. When  $M = 0$ , the circuit is an adder, and when  $M = 1$ , the circuit becomes a subtractor. Each exclusive-OR gate receives input  $M$  and one of the inputs of  $B$ . When  $M = 0$ , we have  $B_i = B_i$ . The full adders receive the value of  $B$ , the input carry is 0, and the circuit performs  $A$  plus  $B$ . When  $M = 1$ , we have  $B_i = \bar{B}_i$  and  $C_0 = 1$ . The  $B$  inputs are all complemented and a 1 is added through the input carry. The circuit performs the operation  $A$  plus the 2's complement of  $B$ . (The exclusive-OR with output  $V$  is for detecting an overflow.) It is worth noting that binary numbers in the signed-complement system are added and subtracted by the same basic addition and subtraction rules as are unsigned numbers. Therefore, computers need only one common hardware circuit to handle both types of arithmetic. The user or programmer must interpret the results of such addition or subtraction differently, depending on whether it is assumed that the numbers are signed or unsigned.

### Overflow:

When two numbers with  $n$  digits each are added and the sum is a number occupying  $n + 1$  digits, we say that an overflow occurred. This is true for binary or decimal numbers, signed or unsigned. When the addition is performed with paper and pencil, an overflow is not a problem, since there is no limit by the width of the page to write down the sum. Overflow is a problem in digital computers because the number of bits that hold the number is finite and a result that contains  $n + 1$  bits cannot be accommodated by an  $n$ -bit word. For this reason, many computers

detect the occurrence of an overflow, and when it occurs, a corresponding flip-flop is set that can then be checked by the user. The detection of an overflow after the addition of two binary numbers depends on whether the numbers are considered to be signed or unsigned. When two unsigned numbers are added, an overflow is detected from the end carry out of the most significant position. In the case of signed numbers, two details are important: the leftmost bit always represents the sign, and negative numbers are in 2's-complement form. When two signed numbers are added, the sign bit is treated as part of the number and the end carry does not indicate an overflow. An overflow cannot occur after an addition if one number is positive and the other is negative, since adding a positive number to a negative number produces a result whose magnitude is smaller than the larger of the two original numbers. An overflow may occur if the two numbers added are both positive or both negative. To see how this can happen, consider the following example: Two signed binary numbers, +70 and +80, are stored in two eight-bit registers. The range of numbers that each register can accommodate is from binary +127 to binary -128. Since the sum of the two numbers is +150, it exceeds the capacity of an eight-bit register. This is also true for -70 and -80.

|          |           |          |           |
|----------|-----------|----------|-----------|
| carries: | 0 1       | carries: | 1 0       |
| +70      | 0 1000110 | -70      | 1 0111010 |
| +80      | 0 1010000 | -80      | 1 0110000 |
| <hr/>    | <hr/>     | <hr/>    | <hr/>     |
| +150     | 1 0010110 | -150     | 0 1101010 |

The binary adder-subtractor circuit with outputs C and V is shown in Fig. 4.13 . If the two binary numbers are considered to be unsigned, then the C bit detects a carry after addition or a borrow after subtraction. If the numbers are considered to be signed, then the V bit detects an overflow. If  $V = 0$  after an addition or subtraction, then no overflow occurred and the  $n$ -bit result is correct. If  $V = 1$ , then the result of the operation contains  $n + 1$  bits, but only the rightmost  $n$  bits of the number fit in the space available, so an overflow has occurred. The  $1n + 12$ th bit is the actual sign and has been shifted out of position.

#### Decimal Adder:

Computers or calculators that perform arithmetic operations directly in the decimal number system represent decimal numbers in binary coded form. An adder for such a computer must employ arithmetic circuits that accept coded decimal numbers and present results in the same code. For binary addition, it is sufficient to consider a pair of significant bits together with a previous carry. A decimal adder requires a minimum of nine inputs and five outputs, since four bits are required to code each decimal digit and the circuit must have an input and output carry. There is a wide variety of possible decimal adder circuits, depending upon the code used to represent the decimal digits. Here we examine a decimal adder for the BCD code. BCD Adder Consider the arithmetic addition of two decimal digits in BCD, together with an input carry from a previous stage. Since each input digit does not exceed 9, the output sum cannot be greater than  $9 + 9 + 1 = 19$ , the 1 in the sum being an input carry. Suppose we apply two BCD digits to a four-bit binary adder. The adder will form the sum in binary and produce a result that ranges from 0 through 19. These binary numbers are listed in Table 4.5 and are labeled by

symbols K, Z<sub>8</sub>, Z<sub>4</sub>, Z<sub>2</sub>, and Z<sub>1</sub>. K is the carry, and the subscripts under the letter Z represent the weights 8, 4, 2, and 1 that can be assigned to the four bits in the BCD code. The columns under the binary sum list the binary value that appears in the outputs of the four-bit binary adder. The output sum of two decimal digits must be represented in BCD and should appear in the form listed in the columns under “BCD Sum.” The problem is to find a rule by which the binary sum is converted to the correct BCD digit representation of the number in the BCD sum. In examining the contents of the table, it becomes apparent that when the binary sum is equal to or less than 1001, the corresponding BCD number is identical, and therefore no conversion is needed. When the binary sum is greater than 1001, we obtain an invalid BCD representation. The addition of binary 6 (0110) to the binary sum converts it to the correct BCD representation and also produces an output carry as required.

**Table 4.5**  
*Derivation of BCD Adder*

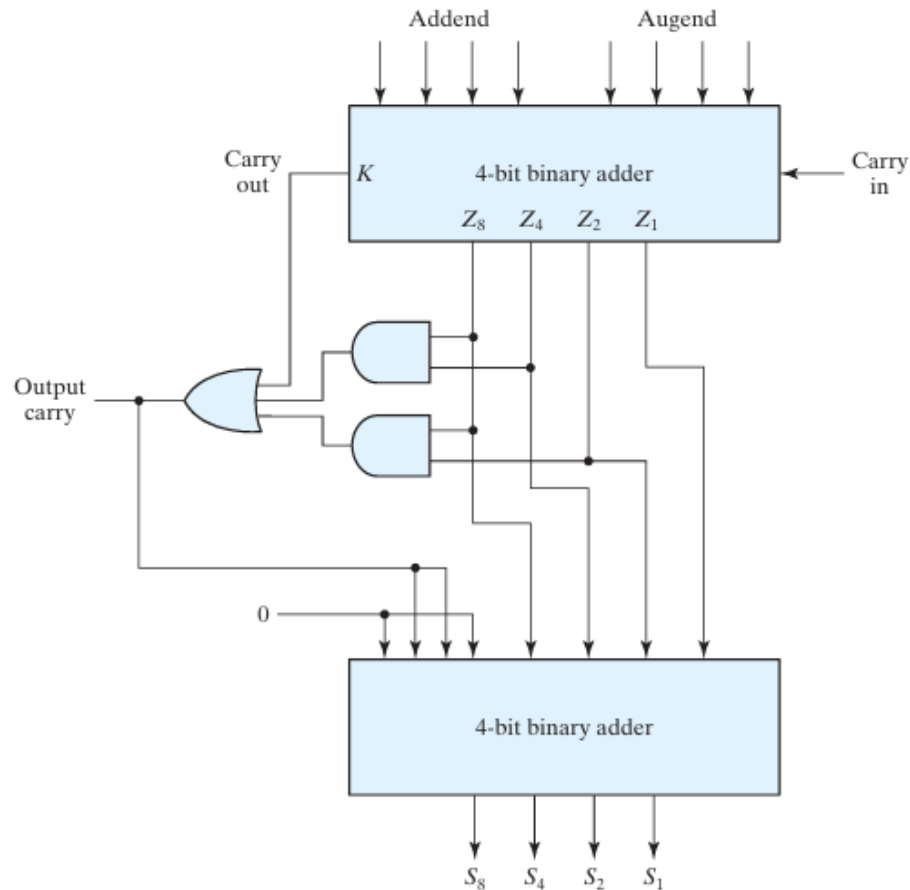
| Binary Sum |                |                |                |                | BCD Sum |                |                |                |                | Decimal |
|------------|----------------|----------------|----------------|----------------|---------|----------------|----------------|----------------|----------------|---------|
| K          | Z <sub>8</sub> | Z <sub>4</sub> | Z <sub>2</sub> | Z <sub>1</sub> | C       | S <sub>8</sub> | S <sub>4</sub> | S <sub>2</sub> | S <sub>1</sub> |         |
| 0          | 0              | 0              | 0              | 0              | 0       | 0              | 0              | 0              | 0              | 0       |
| 0          | 0              | 0              | 0              | 1              | 0       | 0              | 0              | 0              | 1              | 1       |
| 0          | 0              | 0              | 1              | 0              | 0       | 0              | 0              | 1              | 0              | 2       |
| 0          | 0              | 0              | 1              | 1              | 0       | 0              | 0              | 1              | 1              | 3       |
| 0          | 0              | 1              | 0              | 0              | 0       | 0              | 1              | 0              | 0              | 4       |
| 0          | 0              | 1              | 0              | 1              | 0       | 0              | 1              | 0              | 1              | 5       |
| 0          | 0              | 1              | 1              | 0              | 0       | 0              | 1              | 1              | 0              | 6       |
| 0          | 0              | 1              | 1              | 1              | 0       | 0              | 1              | 1              | 1              | 7       |
| 0          | 1              | 0              | 0              | 0              | 0       | 1              | 0              | 0              | 0              | 8       |
| 0          | 1              | 0              | 0              | 1              | 0       | 1              | 0              | 0              | 1              | 9       |
| 0          | 1              | 0              | 1              | 0              | 1       | 0              | 0              | 0              | 0              | 10      |
| 0          | 1              | 0              | 1              | 1              | 1       | 0              | 0              | 0              | 1              | 11      |
| 0          | 1              | 1              | 0              | 0              | 1       | 0              | 0              | 1              | 0              | 12      |
| 0          | 1              | 1              | 0              | 1              | 1       | 0              | 0              | 1              | 1              | 13      |
| 0          | 1              | 1              | 1              | 0              | 1       | 0              | 1              | 0              | 0              | 14      |
| 0          | 1              | 1              | 1              | 1              | 1       | 0              | 1              | 0              | 1              | 15      |
| 1          | 0              | 0              | 0              | 0              | 1       | 0              | 1              | 1              | 0              | 16      |
| 1          | 0              | 0              | 0              | 1              | 1       | 0              | 1              | 1              | 1              | 17      |
| 1          | 0              | 0              | 1              | 0              | 1       | 1              | 0              | 0              | 0              | 18      |
| 1          | 0              | 0              | 1              | 1              | 1       | 1              | 0              | 0              | 1              | 19      |

The logic circuit that detects the necessary correction can be derived from the entries in the table. It is obvious that a correction is needed when the binary sum has an output carry K=1. The other six combinations from 1010 through 1111 that need a correction have a 1 in position Z<sub>8</sub>. To distinguish them from binary 1000 and 1001, which also have a 1 in position Z<sub>8</sub>, we specify further that either Z<sub>4</sub> or Z<sub>2</sub> must have a 1. The condition for a correction and an output carry can be expressed by the Boolean function  $C = K + Z_8Z_4 + Z_8Z_2$ .

When C=1, it is necessary to add 0110 to the binary sum and provide an output carry for the next stage. A BCD adder that adds two BCD digits and produces a sum digit in BCD is shown in Fig. 4.14 . The two decimal digits, together with the input carry, are first added in the top four-bit adder to produce the binary sum. When the output carry is equal to 0, nothing is added to the binary sum. When it is equal to 1, binary 0110 is added to the binary sum through the



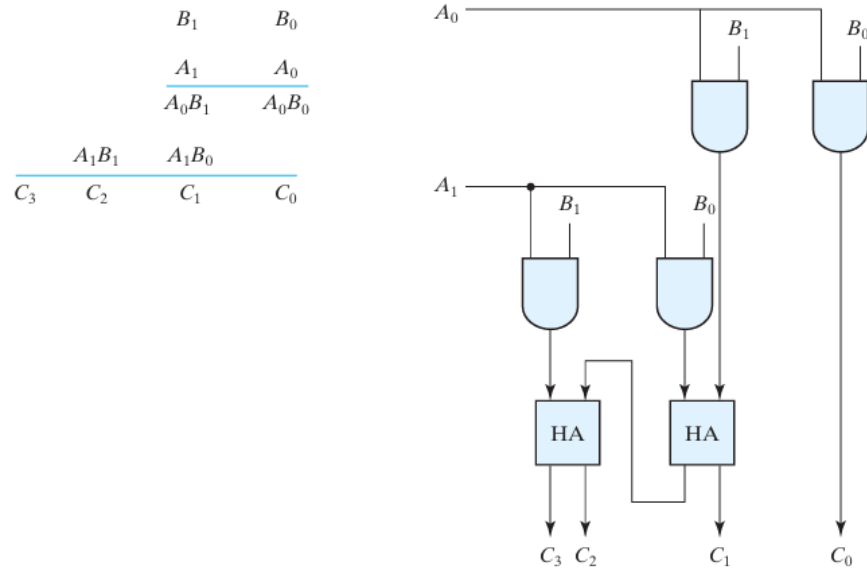
bottom four-bit adder. The output carry generated from the bottom adder can be ignored, since it supplies information already available at the output carry terminal. A decimal parallel adder that adds  $n$  decimal digits needs  $n$  BCD adder stages. The output carry from one stage must be connected to the input carry of the next higher order stage.



**FIGURE 4.14**  
Block diagram of a BCD adder

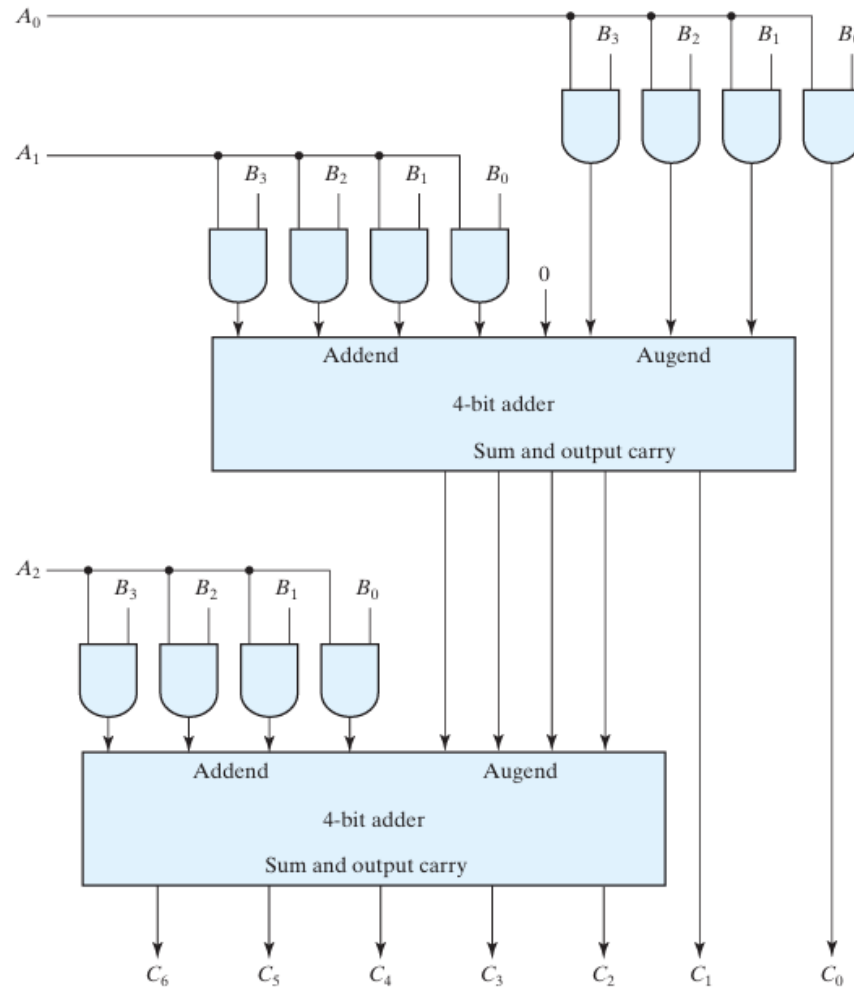
### Binary Multiplier:

The multiplicand is multiplied by each bit of the multiplier, starting from the least significant bit. Each such multiplication forms a partial product. Successive partial products are shifted one position to the left. The final product is obtained from the sum of the partial products. To see how a binary multiplier can be implemented with a combinational circuit, consider the multiplication of two 2-bit numbers as shown in Fig. 4.15. The multiplicand bits are  $B_1$  and  $B_0$ , the multiplier bits are  $A_1$  and  $A_0$ , and the product is  $C_3C_2C_1C_0$ . The first partial product is formed by multiplying  $B_1B_0$  by  $A_0$ . The multiplication of two bits such as  $A_0$  and  $B_0$  produces a 1 if both bits are 1; otherwise, it produces a 0. This is identical to an AND operation. Therefore, the partial product can be implemented with AND gates as shown in the diagram. The second partial product is formed by multiplying  $B_1B_0$  by  $A_1$  and shifting one position to the left. The two partial products are added with two half-adder (HA) circuits. Usually, there are more bits in the partial products and it is necessary to use full adders to produce the sum of the partial products. Note that the least significant bit of the product does not have to go through an adder, since it is formed by the output of the first AND gate.



**FIGURE 4.15**  
Two-bit by two-bit binary multiplier

similar fashion. A bit of the multiplier is ANDed with each bit of the multiplicand in as many levels as there are bits in the multiplier. The binary output in each level of AND gates is added with the partial product of the previous level to form a new partial product. The last level produces the product. For  $J$  multiplier bits and  $K$  multiplicand bits, we need  $1J \times K/2$  AND gates and  $1J - 1/2 K$  -bit adders to produce a product of  $(J + K)$  bits. As a second example, consider a multiplier circuit that multiplies a binary number represented by four bits by a number represented by three bits. Let the multiplicand be represented by  $B_3B_2B_1B_0$  and the multiplier by  $A_2A_1A_0$ . Since  $K = 4$  and  $J = 3$ , we need 12 AND gates and two 4-bit adders to produce a product of seven bits. The logic diagram of the multiplier is shown in Fig. 4.16 .



**FIGURE 4.16**  
Four-bit by three-bit binary multiplier

### Magnitude Comparators:

The comparison of two numbers is an operation that determines whether one number is greater than, less than, or equal to the other number. A magnitude comparator is a combinational circuit that compares two numbers  $A$  and  $B$  and determines their relative magnitudes. The outcome of the comparison is specified by three binary variables that indicate whether  $A > B$ ,  $A = B$ , or  $A < B$ . On the one hand, the circuit for comparing two  $n$ -bit numbers has  $2^{2n}$  entries in the truth table and becomes too cumbersome, even with  $n = 3$ . On the other hand, as one may suspect, a comparator circuit possesses a certain amount of regularity. Digital functions that possess an inherent well-defined regularity can usually be designed by means of an algorithm—a procedure which specifies a finite set of steps that, if followed, give the solution to a problem. We illustrate this method here by deriving an algorithm for the design of a four-bit magnitude comparator. The algorithm is a direct application of the procedure a person uses to compare the relative magnitudes of two numbers. Consider two numbers,  $A$  and  $B$ , with four digits each. Write the coefficients of the numbers in descending order of significance:

$$A = A_3 A_2 A_1 A_0$$

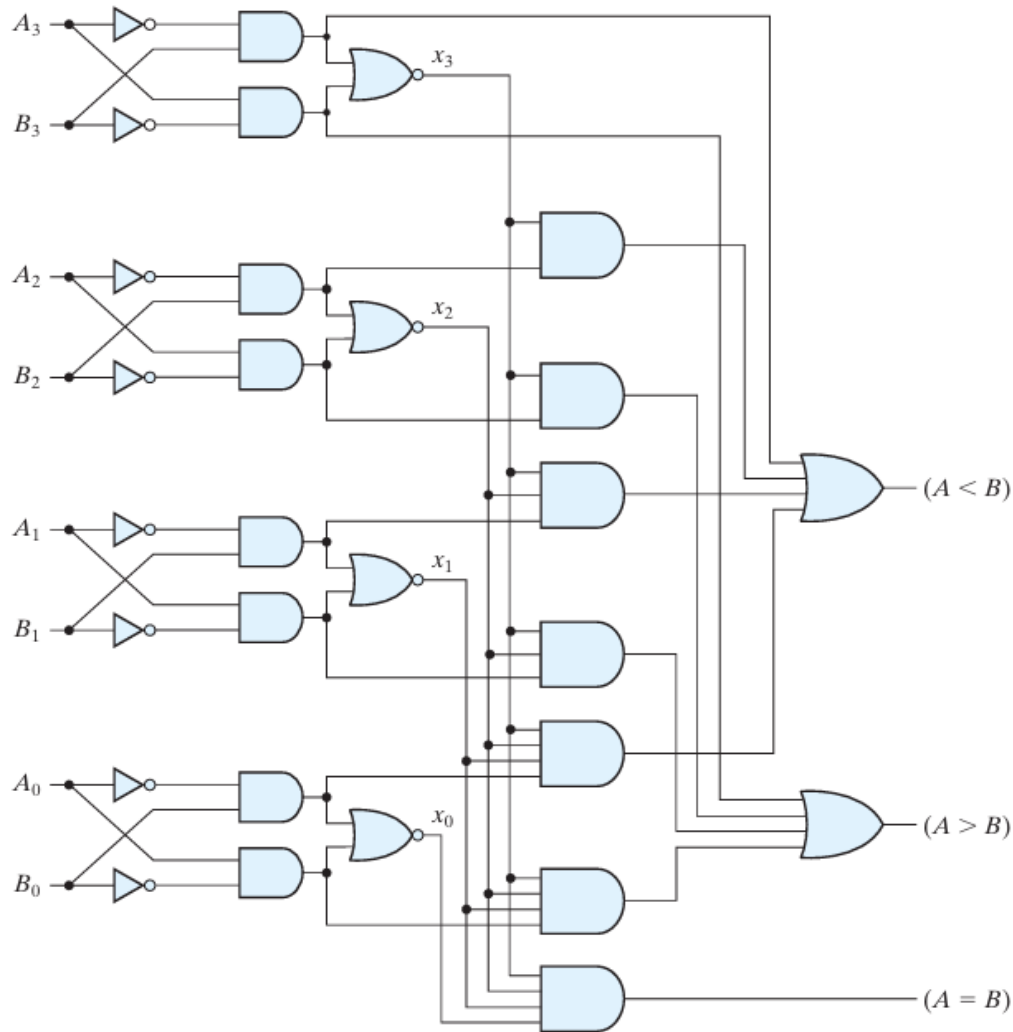
$$B = B_3 B_2 B_1 B_0$$

Each subscripted letter represents one of the digits in the number. The two numbers are equal if all pairs of significant digits are equal:  $A_3 = B_3$ ,  $A_2 = B_2$ ,  $A_1 = B_1$ , and  $A_0 = B_0$ . When the numbers are binary, the digits are either 1 or 0, and the equality of each pair of bits can be expressed logically with an exclusive-NOR function. The equality of the two numbers A and B is displayed in a combinational circuit by an output binary variable that we designate by the symbol  $1A = B_2$ . This binary variable is equal to 1 if the input numbers, A and B, are equal, and is equal to 0 otherwise. The binary variable  $1A = B_2$  is equal to 1 only if all pairs of digits of the two numbers are equal. To determine whether A is greater or less than B, we inspect the relative magnitudes of pairs of significant digits, starting from the most significant position. If the two digits of a pair are equal, we compare the next lower significant pair of digits. The comparison continues until a pair of unequal digits is reached. If the corresponding digit of A is 1 and that of B is 0, we conclude that  $A > B$ . If the corresponding digit of A is 0 and that of B is 1, we have  $A < B$ . The sequential comparison can be expressed logically by the two Boolean functions

$$(A > B) = A_3B'_3 + x_3A_2B'_2 + x_3x_2A_1B'_1 + x_3x_2x_1A_0B'_0$$

$$(A < B) = A'_3B_3 + x_3A'_2B_2 + x_3x_2A'_1B_1 + x_3x_2x_1A'_0B_0$$

The gate implementation of the three output variables just derived is simpler than it seems because it involves a certain amount of repetition. The unequal outputs can use the same gates that are needed to generate the equal output. The logic diagram of the four-bit magnitude comparator is shown in Fig. 4.17. The four x outputs are generated with exclusive-NOR circuits and are applied to an AND gate to give the output binary variable  $1A = B_2$ . The other two outputs use the x variables to generate the Boolean functions listed previously. This is a multilevel implementation and has a regular pattern. The procedure for obtaining magnitude comparator circuits for binary numbers with more than four bits is obvious from this example.

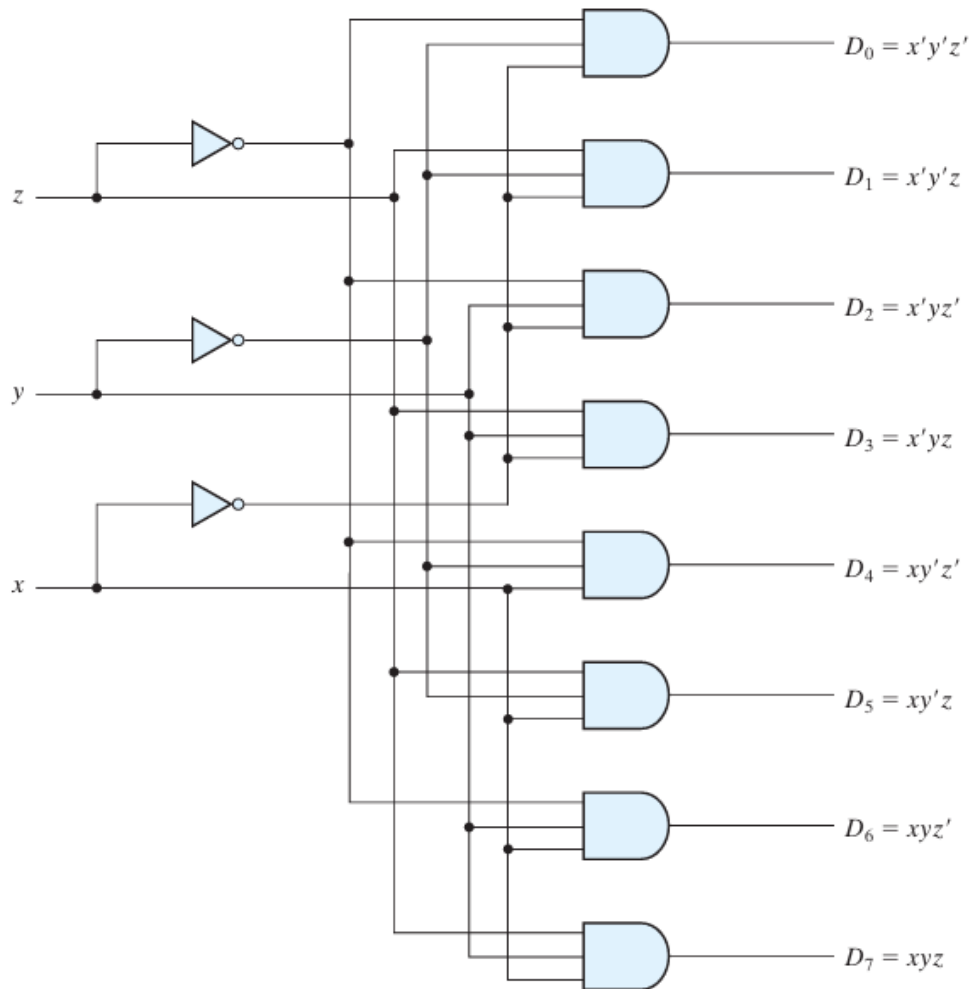


**FIGURE 4.17**  
 Four-bit magnitude comparator

Decoders:

Discrete quantities of information are represented in digital systems by binary codes. A binary code of  $n$  bits is capable of representing up to  $2^n$  distinct elements of coded information. A decoder is a combinational circuit that converts binary information from  $n$  input lines to a maximum of  $2^n$  unique output lines. If the  $n$ -bit coded information has unused combinations, the decoder may have fewer than  $2^n$  outputs. The decoders presented here are called  $n$ -to- $m$ -line decoders, where  $m \leq 2^n$ . Their purpose is to generate the  $2^n$  (or fewer) minterms of  $n$  input variables. Each combination of inputs will assert a unique output. The name decoder is also used in conjunction with other code converters, such as a BCD-to-seven-segment decoder. As an example, consider the three-to-eight-line decoder circuit of Fig. 4.18. The three inputs are decoded into eight outputs, each representing one of the minterms of the three input variables. The three inverters provide the complement of the inputs, and each one of the eight AND gates generates one of the minterms. A particular application of this decoder is binary-to-octal conversion. The input variables represent a binary number, and the outputs represent

the eight digits of a number in the octal number system. However, a three-to-eight-line decoder can be used for decoding any three-bit code to provide eight outputs, one for each element of the code.



**FIGURE 4.18**  
Three-to-eight-line decoder

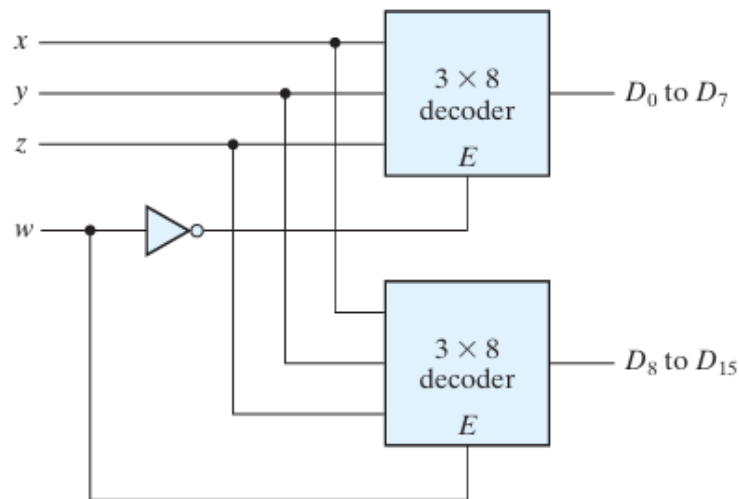
The operation of the decoder may be clarified by the truth table listed in Table 4.6 . For each possible input combination, there are seven outputs that are equal to 0 and only one that is equal to 1. The output whose value is equal to 1 represents the minterm equivalent of the binary number currently available in the input lines. Some decoders are constructed with NAND gates. Since a NAND gate produces the AND operation with an inverted output, it becomes more economical to generate the decoder minterms in their complemented form. Furthermore, decoders include one or more enable inputs to control the circuit operation. A two-to-four-line decoder with an enable input constructed with NAND gates is shown in Fig. 4.19 . The circuit

operates with complemented outputs and a complement enable input. The decoder is enabled when E is equal to 0 (i.e., active-low enable).

**Table 4.6**  
*Truth Table of a Three-to-Eight-Line Decoder*

| Inputs   |          |          | Outputs               |                       |                       |                       |                       |                       |                       |                       |
|----------|----------|----------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| <i>x</i> | <i>y</i> | <i>z</i> | <i>D</i> <sub>0</sub> | <i>D</i> <sub>1</sub> | <i>D</i> <sub>2</sub> | <i>D</i> <sub>3</sub> | <i>D</i> <sub>4</sub> | <i>D</i> <sub>5</sub> | <i>D</i> <sub>6</sub> | <i>D</i> <sub>7</sub> |
| 0        | 0        | 0        | 1                     | 0                     | 0                     | 0                     | 0                     | 0                     | 0                     | 0                     |
| 0        | 0        | 1        | 0                     | 1                     | 0                     | 0                     | 0                     | 0                     | 0                     | 0                     |
| 0        | 1        | 0        | 0                     | 0                     | 1                     | 0                     | 0                     | 0                     | 0                     | 0                     |
| 0        | 1        | 1        | 0                     | 0                     | 0                     | 1                     | 0                     | 0                     | 0                     | 0                     |
| 1        | 0        | 0        | 0                     | 0                     | 0                     | 0                     | 1                     | 0                     | 0                     | 0                     |
| 1        | 0        | 1        | 0                     | 0                     | 0                     | 0                     | 0                     | 1                     | 0                     | 0                     |
| 1        | 1        | 0        | 0                     | 0                     | 0                     | 0                     | 0                     | 0                     | 1                     | 0                     |
| 1        | 1        | 1        | 0                     | 0                     | 0                     | 0                     | 0                     | 0                     | 0                     | 1                     |

only one output can be equal to 0 at any given time; all other outputs are equal to 1. The output whose value is equal to 0 represents the minterm selected by inputs A and B. The circuit is disabled when E is equal to 1, regardless of the values of the other two inputs. When the circuit is disabled, none of the outputs are equal to 0 and none of the minterms are selected. In general, a decoder may operate with complemented or uncomplemented outputs. The enable input may be activated with a 0 or with a 1 signal. Some decoders have two or more enable inputs that must satisfy a given logic condition in order to enable the circuit. A decoder with enable input can function as a demultiplexer—a circuit that receives information from a single line and directs it to one of  $2^n$  possible output lines. The selection of a specific output is controlled by the bit combination of  $n$  selection lines. The decoder of Fig. 4.19 can function as a one-to-four-line demultiplexer when E is taken as a data input line and A and B are taken as the selection inputs. The single input variable E has a path to all four outputs, but the input information is directed to only one of the output lines, as specified by the binary combination of the two selection lines A and B. This feature can be verified from the truth table of the circuit. For example, if the selection lines  $AB = 10$ , output  $D_2$  will be the same as the input value E, while all other outputs are maintained at 1. Because decoder and demultiplexer operations are obtained from the same circuit, a decoder with an enable input is referred to as a decoder – demultiplexer. Decoders with enable inputs can be connected together to form a larger decoder circuit. Figure 4.20 shows two 3-to-8-line decoders with enable inputs connected to form a 4-to-16-line decoder. When  $w = 0$ , the top decoder is enabled and the other is disabled. The bottom decoder outputs are all 0's, and the top eight outputs generate minterms 0000 to 0111. When  $w = 1$ , the enable conditions are reversed: The bottom decoder outputs generate minterms 1000 to 1111, while the outputs of the top decoder are all 0's. This example demonstrates the usefulness of enable inputs in decoders and other combinational logic components. In general, enable inputs are a convenient feature for interconnecting two or more standard components for the purpose of combining them into a similar function with more inputs and outputs.



**FIGURE 4.20**  
**4 × 16 decoder constructed with two 3 × 8 decoders**

### Combinational Logic Implementation

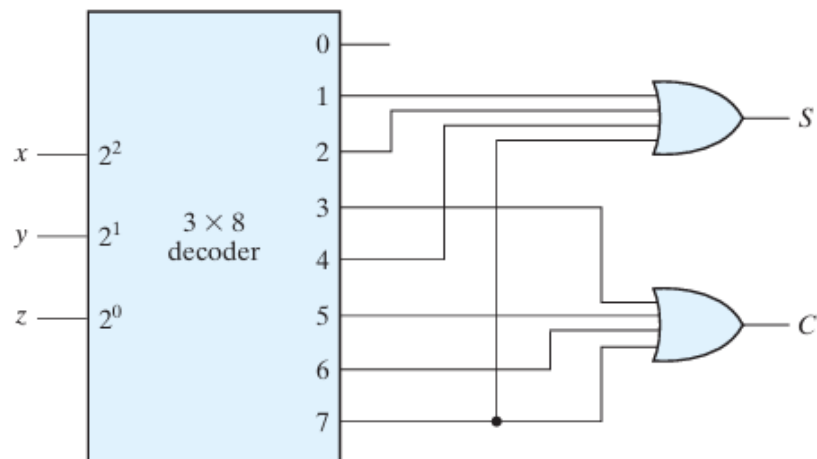
A decoder provides the  $2^n$  minterms of  $n$  input variables. Each asserted output of the decoder is associated with a unique pattern of input bits. Since any Boolean function can be expressed in sum-of-minterms form, a decoder that generates the minterms of the function, together with an external OR gate that forms their logical sum, provides a hardware implementation of the function. In this way, any combinational circuit with  $n$  inputs and  $m$  outputs can be implemented with an  $n$ -to- $2^n$ -line decoder and  $m$  OR gates. The procedure for implementing a combinational circuit by means of a decoder and OR gates requires that the Boolean function for the circuit be expressed as a sum of minterms. A decoder is then chosen that generates all the minterms of the input variables. The inputs to each OR gate are selected from the decoder outputs according to the list of minterms of each function. This procedure will be illustrated by an example that implements a full-adder circuit. From the truth table of the full adder (see Table 4.4), we obtain the functions for the combinational circuit in sum-of-minterms form:

$$S(x, y, z) = \sum(1, 2, 4, 7)$$

$$C(x, y, z) = \sum(3, 5, 6, 7)$$

A function with a long list of minterms requires an OR gate with a large number of inputs. A function having a list of  $k$  minterms can be expressed in its complemented form  $F$  with  $2^n - k$  minterms. If the number of minterms in the function is greater than  $2^{n-1}$ , then  $F$  can be expressed with fewer minterms. In such a case, it is advantageous to use a NOR gate to sum the minterms of  $F$ . The output of the NOR gate complements this sum and generates the normal output  $F$ . If NAND gates are used for the decoder, as in Fig. 4.19, then the external gates must be NAND gates instead of OR gates. This is because a two-level NAND gate circuit implements a sum-of-minterms function and is equivalent to a two-level AND–OR circuit.





**FIGURE 4.21**  
 Implementation of a full adder with a decoder

## Encoders

An encoder is a digital circuit that performs the inverse operation of a decoder. An encoder has  $2^n$  (or fewer) input lines and  $n$  output lines. The output lines, as an aggregate, generate the binary code corresponding to the input value. An example of an encoder is the octal-to-binary encoder whose truth table is given in Table 4.7. It has eight inputs (one for each of the octal digits) and three outputs that generate the corresponding binary number. It is assumed that only one input has a value of 1 at any given time. The encoder can be implemented with OR gates whose inputs are determined directly from the truth table. Output  $z$  is equal to 1 when the input octal digit is 1, 3, 5, or 7. Output  $y$  is 1 for octal digits 2, 3, 6, or 7, and output  $x$  is 1 for digits 4, 5, 6, or 7. These conditions can be expressed by the following Boolean output functions:

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

**Table 4.7**  
*Truth Table of an Octal-to-Binary Encoder*

| Inputs |       |       |       |       |       |       |       | Outputs |     |     |
|--------|-------|-------|-------|-------|-------|-------|-------|---------|-----|-----|
| $D_0$  | $D_1$ | $D_2$ | $D_3$ | $D_4$ | $D_5$ | $D_6$ | $D_7$ | $x$     | $y$ | $z$ |
| 1      | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0       | 0   | 0   |
| 0      | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0       | 0   | 1   |
| 0      | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0       | 1   | 0   |
| 0      | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0       | 1   | 1   |
| 0      | 0     | 0     | 0     | 1     | 0     | 0     | 0     | 1       | 0   | 0   |
| 0      | 0     | 0     | 0     | 0     | 1     | 0     | 0     | 1       | 0   | 1   |
| 0      | 0     | 0     | 0     | 0     | 0     | 1     | 0     | 1       | 1   | 0   |
| 0      | 0     | 0     | 0     | 0     | 0     | 0     | 1     | 1       | 1   | 1   |

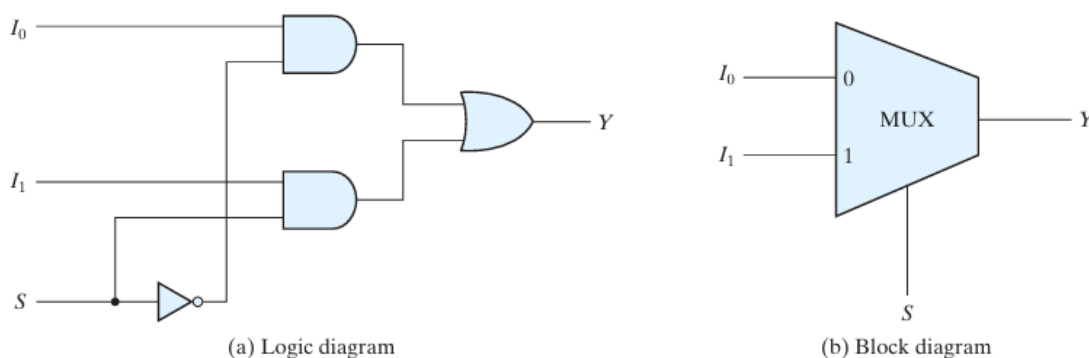
The encoder defined in Table 4.7 has the limitation that only one input can be active at any given time. If two inputs are active simultaneously, the output produces an undefined combination. For example, if  $D_3$  and  $D_6$  are 1 simultaneously, the output of the encoder will be 111 because all three outputs are equal to 1. The output 111 does not represent either binary 3 or binary 6. To resolve this ambiguity, encoder circuits must establish an input priority to ensure that only one input is encoded. If we establish a higher priority for inputs with higher subscript numbers, and if both  $D_3$  and  $D_6$  are 1 at the same time, the output will be 110 because  $D_6$  has higher priority than  $D_3$ . Another ambiguity in the octal-to-binary encoder is that an output with all 0's is generated when all the inputs are 0; but this output is the same as when  $D_0$  is equal to 1. The discrepancy can be resolved by providing one more output to indicate whether at least one input is equal to 1.

### Priority Encoder

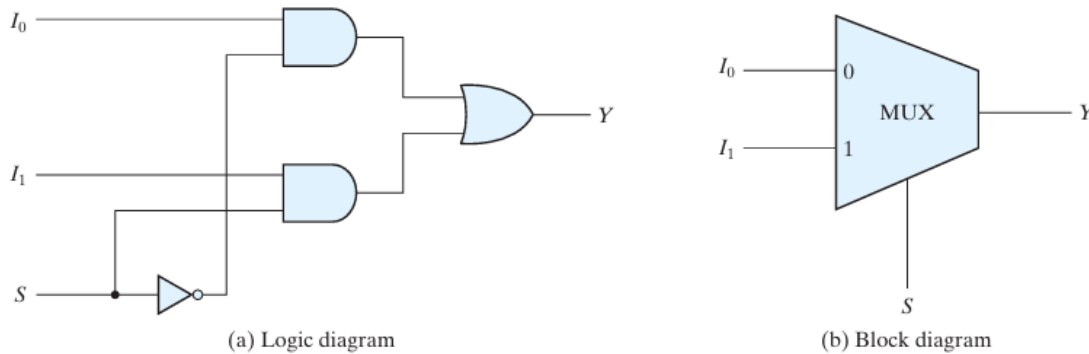
A priority encoder is an encoder circuit that includes the priority function. The operation of the priority encoder is such that if two or more inputs are equal to 1 at the same time, the input having the highest priority will take precedence. The truth table of a four-input priority encoder is given in Table 4.8. In addition to the two outputs  $x$  and  $y$ , the circuit has a third output designated by  $V$ ; this is a valid bit indicator that is set to 1 when one or more inputs are equal to 1. If all inputs are 0, there is no valid input and  $V$  is equal to 0. The other two outputs are not inspected when  $V$  equals 0 and are specified as don't-care conditions. Note that whereas  $X$ 's in output columns represent don't-care conditions, the  $X$ 's in the input columns are useful for representing a truth table in condensed form. Instead of listing all 16 minterms of four variables, the truth table uses an  $X$  to represent either 1 or 0. For example,  $X100$  represents the two minterms 0100 and 1100. According to Table 4.8, the higher the subscript number, the higher the priority of the input. Input  $D_3$  has the highest priority, so, regardless of the values of the other inputs, when this input is 1, the output for  $xy$  is 11 (binary 3).  $D_2$  has the next priority level. The output is 10 if  $D_2=1$ , provided that  $D_3=0$ , regardless of the values of the other two lower priority inputs. The output for  $D_1$  is generated only if higher priority inputs are 0, and so on down the priority levels.

## Multiplexer :

A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of selection lines. Normally, there are  $2^n$  input lines and  $n$  selection lines whose bit combinations determine which input is selected. A two-to-one-line multiplexer connects one of two 1-bit sources to a common destination, as shown in Fig. 4.24 . The circuit has two data input lines, one output line, and one selection line  $S$  . When  $S = 0$ , the upper AND gate is enabled and  $I_0$  has a path to the output. When  $S = 1$ , the lower AND gate is enabled and  $I_1$  has a path to the output. The multiplexer acts like an electronic switch that selects one of two sources. The block diagram of a multiplexer is sometimes depicted by a wedge-shaped symbol, as shown in Fig. 4.24(b) . It suggests visually how a selected one of multiple data sources is directed into a single destination. The multiplexer is often labeled “MUX” in block diagrams. A four-to-one-line multiplexer is shown in Fig. 4.25 . Each of the four inputs,  $I_0$  through  $I_3$ , is applied to one input of an AND gate. Selection lines  $S_1$  and  $S_0$  are decoded to select a particular AND gate. The outputs of the AND gates are applied to a single OR gate that provides the one-line output. The function table lists the input that is passed to the output for each combination of the binary selection values. To demonstrate the operation of the circuit, consider the case when  $S_1S_0 = 10$ . The AND gate associated with input  $I_2$  has two of its inputs equal to 1 and the third input connected to  $I_2$ . The other three AND gates have at least one input equal to 0, which makes their outputs equal to 0. The output of the OR gate is now equal to the value of  $I_2$ , providing a path from the selected input to the output. A multiplexer is also called a data selector , since it selects one of many inputs and steers the binary information to the output line. The AND gates and inverters in the multiplexer resemble a decoder circuit, and indeed, they decode the selection input lines. In general, a  $2^n$ -to-1-line multiplexer is constructed from an  $n$ -to- $2^n$  decoder by adding  $2^n$  input lines to it, one to each AND gate. The outputs of the AND gates are applied to a single OR gate. The size of a multiplexer is specified by



**FIGURE 4.24**  
Two-to-one-line multiplexer



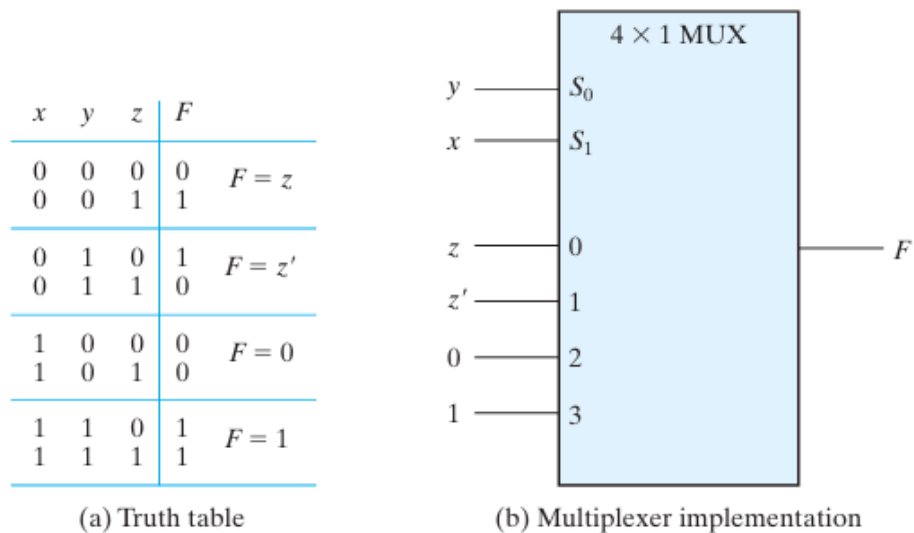
**FIGURE 4.24**  
Two-to-one-line multiplexer

the number  $2^n$  of its data input lines and the single output line. The  $n$  selection lines are implied from the  $2^n$  data lines. As in decoders, multiplexers may have an enable input to control the operation of the unit. When the enable input is in the inactive state, the outputs are disabled, and when it is in the active state, the circuit functions as a normal multiplexer. Multiplexer circuits can be combined with common selection inputs to provide multiple-bit selection logic. As an illustration, a quadruple 2-to-1-line multiplexer is shown in Fig. 4.26. The circuit has four multiplexers, each capable of selecting one of two input lines. Output  $Y_0$  can be selected to come from either input  $A_0$  or input  $B_0$ . Similarly, output  $Y_1$  may have the value of  $A_1$  or  $B_1$ , and so on. Input selection line  $S$  selects one of the lines in each of the four multiplexers. The enable input  $E$  must be active (i.e., asserted) for normal operation. Although the circuit contains four 2-to-1-line multiplexers, we are more likely to view it as a circuit that selects one of two 4-bit sets of data lines. As shown in the function table, the unit is enabled when  $E = 0$ . Then, if  $S = 0$ , the four  $A$  inputs have a path to the four outputs. If, by contrast,  $S = 1$ , the four  $B$  inputs are applied to the outputs. The outputs have all 0's when  $E = 1$ , regardless of the value of  $S$ .

### Boolean Function Implementation

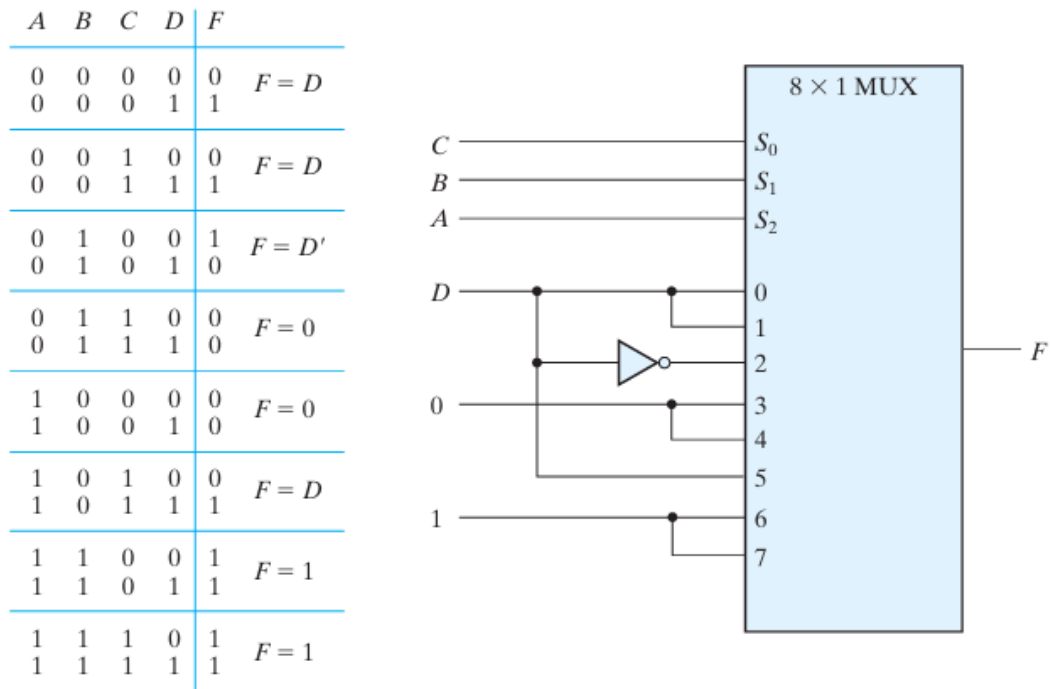
It was shown that a decoder can be used to implement Boolean functions by employing external OR gates. An examination of the logic diagram of a multiplexer reveals that it is essentially a decoder that includes the OR gate within the unit. The minterms of a function are generated in a multiplexer by the circuit associated with the selection inputs. The individual minterms can be selected by the data inputs, thereby providing a method of implementing a Boolean function of  $n$  variables with a multiplexer that has  $n$  selection inputs and  $2^n$  data inputs, one for each minterm. We will now show a more efficient method for implementing a Boolean function of  $n$  variables with a multiplexer that has  $n-1$  selection inputs. The first  $n-1$  variables of the function are connected to the selection inputs of the multiplexer. The remaining single variable of the function is used for the data inputs. If the single variable is denoted by  $z$ , each data input of the multiplexer will be  $z$ ,  $\bar{z}$ , 1, or 0. To demonstrate this procedure, consider the Boolean function  $F(x, y, z) = (1, 2, 6, 7)$ . This function of three variables can be implemented with a four-to-one-line multiplexer as shown in Fig. 4.27. The two variables  $x$  and  $y$  are applied to the selection lines in that order;  $x$  is connected to the  $S_1$  input and  $y$  to the  $S_0$  input. The values for the data input lines are determined from the truth table of the function. When  $xy = 00$ , output  $F$  is equal to  $z$  because  $F = 0$  when  $z = 0$  and  $F = 1$  when  $z = 1$ . This requires that variable

$z$  be applied to data input 0. The operation of the multiplexer is such that when  $xy = 00$ , data input 0 has a path to the output, and that makes  $F$  equal to  $z$ . In a similar fashion, we can determine the required input to data lines 1, 2, and 3 from the value of  $F$  when  $xy = 01, 10$ , and  $11$ , respectively. This particular example shows all four possibilities that can be obtained for the data inputs. The general procedure for implementing any Boolean function of  $n$  variables with a multiplexer with  $n-1$  selection inputs and  $2^{n-1}$  data inputs follows from the previous example. To begin with, Boolean function is listed in a truth table. Then first  $n-1$  variables in the table are applied to the selection inputs of the multiplexer. For each combination of the selection variables, we evaluate the output as a function of the last variable. This function can be 0, 1, the variable, or the complement of the variable. These values are then applied to the data inputs in the proper order. As a second example, consider the implementation of the Boolean function  $F(A, B, C, D) = (1, 3, 4, 11, 12, 13, 14, 15)$ . This function is implemented with a multiplexer with three selection inputs as shown in Fig. 4.28. Note that the first variable  $A$  must be connected to selection input  $S_2$  so that  $A, B$ , and  $C$  correspond to selection inputs  $S_2, S_1$ , and  $S_0$ , respectively. The values for the data inputs are determined from the truth table listed in the figure. The corresponding data line number is determined from the binary combination of  $ABC$ . For example, the table shows that when  $ABC = 101$ ,  $F = D$ , so the input variable  $D$  is applied to data input 5. The binary constants 0 and 1 correspond to two fixed signal values. When integrated circuits are used, logic 0 corresponds to signal ground and logic 1 is equivalent to the power signal, depending on the technology (e.g., 3 V).



**FIGURE 4.27**

**Implementing a Boolean function with a multiplexer**

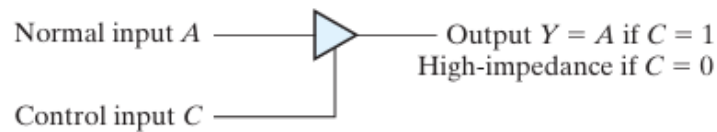


**FIGURE 4.28**  
Implementing a four-input function with a multiplexer

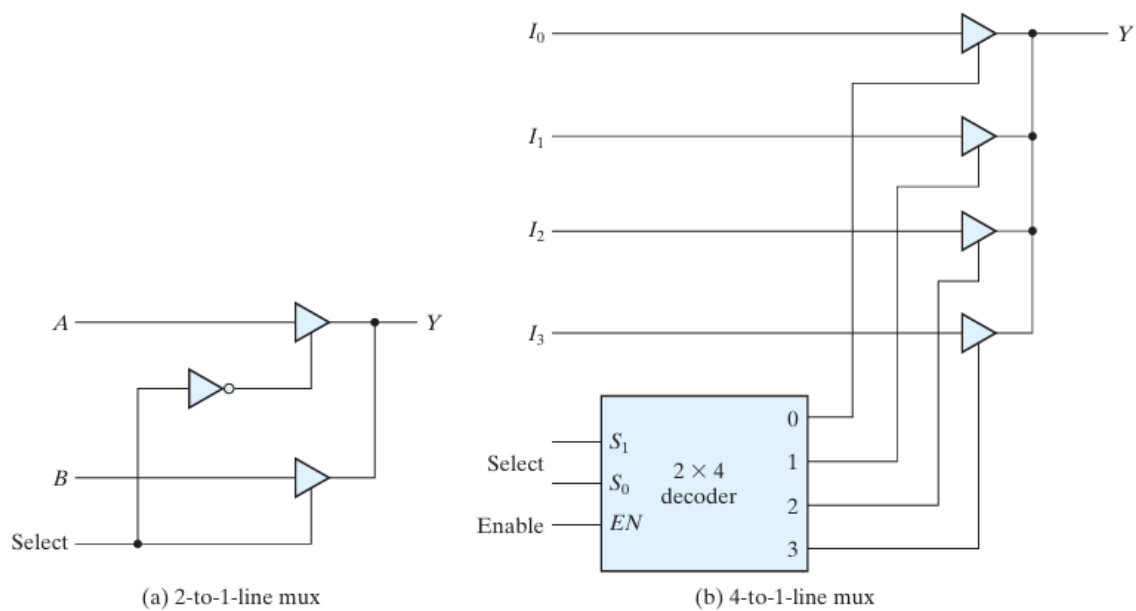
### Three-State Gates

A multiplexer can be constructed with three-state gates—digital circuits that exhibit three states. Two of the states are signals equivalent to logic 1 and logic 0 as in a conventional gate. The third state is a high-impedance state in which (1) the logic behaves like an open circuit, which means that the output appears to be disconnected, (2) the circuit has no logic significance, and (3) the circuit connected to the output of the three-state gate is not affected by the inputs to the gate. Three-state gates may perform any conventional logic, such as AND or NAND. However, the one most commonly used is the buffer gate. The graphic symbol for a three-state buffer gate is shown in Fig. 4.29. It is distinguished from a normal buffer by an input control line entering the bottom of the symbol. The buffer has a normal input, an output, and a control input that determines the state of the output. When the control input is equal to 1, the output is enabled and the gate behaves like a conventional buffer, with the output equal to the normal input. When the control input is 0, the output is disabled and the gate goes to a high-impedance state, regardless of the value in the normal input. The high-impedance state of a three-state gate provides a special feature not available in other gates. Because of this feature, a large number of three-state gate outputs can be connected with wires to form a common line without endangering loading effects. The construction of multiplexers with three-state buffers is demonstrated in Fig. 4.30. Figure 4.30(a) shows the construction of a two-to-one-line multiplexer with 2 three-state buffers and an inverter. The two outputs are connected together to form a single output line. (Note that this type of connection cannot be made with gates that do not have three-state outputs.) When the select input is 0, the upper buffer is enabled by its control input and the lower buffer is disabled. Output Y is then equal to input A. When the select input is 1, the lower buffer is enabled and Y is equal to B. The construction of a four-to-one-line multiplexer is shown in Fig. 4.30(b). The outputs of 4 three-state buffers are

connected together to form a single output line. The control inputs to the buffers determine which one of the four normal inputs  $I_0$  through



**FIGURE 4.29**  
Graphic symbol for a three-state buffer

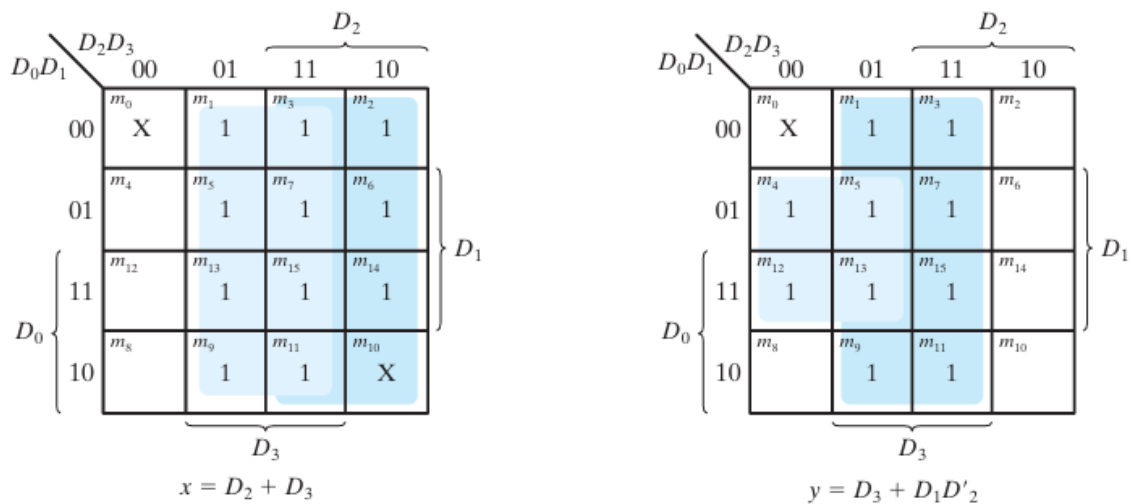


**FIGURE 4.30**  
Multiplexers with three-state gates

$I_3$  will be connected to the output line. No more than one buffer may be in the active state at any given time. The connected buffers must be controlled so that only 1 three state buffer has access to the output while all other buffers are maintained in a high impedance state. One way to ensure that no more than one control input is active at any given time is to use a decoder, as shown in the diagram. When the enable input of the decoder is 0, all of its four outputs are 0 and the bus line is in a high-impedance state because all four buffers are disabled. When the enable input is active, one of the three state buffers will be active, depending on the binary value in the select inputs of the decoder. Careful investigation reveals that this circuit is another way of constructing a four-to-one-line multiplexer.

**Table 4.8**  
*Truth Table of a Priority Encoder*

| Inputs |       |       |       | Outputs |     |     |
|--------|-------|-------|-------|---------|-----|-----|
| $D_0$  | $D_1$ | $D_2$ | $D_3$ | $x$     | $y$ | $V$ |
| 0      | 0     | 0     | 0     | X       | X   | 0   |
| 1      | 0     | 0     | 0     | 0       | 0   | 1   |
| X      | 1     | 0     | 0     | 0       | 1   | 1   |
| X      | X     | 1     | 0     | 1       | 0   | 1   |
| X      | X     | X     | 1     | 1       | 1   | 1   |

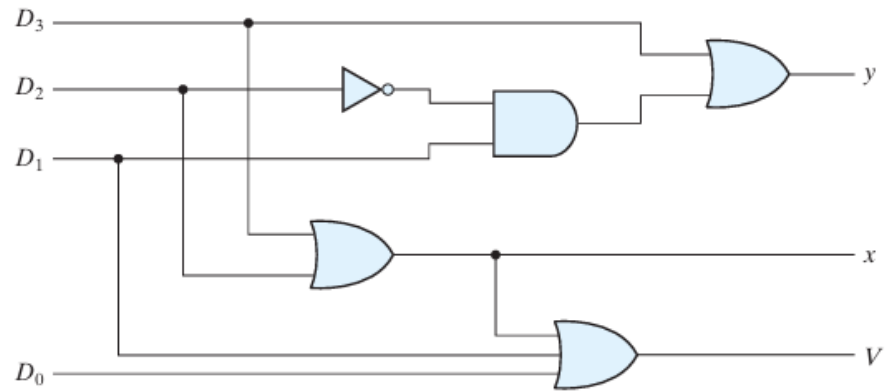


**FIGURE 4.22**  
*Maps for a priority encoder*

The maps for simplifying outputs  $x$  and  $y$  are shown in Fig. 4.22. The minterms for the two functions are derived from Table 4.8. Although the table has only five rows, when each X in a row is replaced first by 0 and then by 1, we obtain all 16 possible input combinations. For example, the fourth row in the table, with inputs XX10, represents the four minterms 0010, 0110, 1010, and 1110. The simplified Boolean expressions for the priority encoder are obtained from the maps. The condition for output  $V$  is an OR function of all the input variables. The priority encoder is implemented in Fig. 4.23 according to the following Boolean functions:



$$\begin{aligned}x &= D_2 + D_3 \\y &= D_3 + D_1 D_2' \\V &= D_0 + D_1 + D_2 + D_3\end{aligned}$$



**FIGURE 4.23**  
Four-input priority encoder